



دانشکده فنی و مهندسی

پروژه نرم افزار کارشناسی مهندسی کامپیوتر

مبانی و مفاهیم برنامه نویسی گرافیک

نگارش:

امیرمحمد خالقی فرید

استاد:

امید آقا لطیفی

تاریخ: آذر 1404

**IN THE NAME OF
GOD**

تقدیم به

همه افراد و پتانسیل هایی که مدفون شدن تحت قید و بند مشکلاتی که مقصرش نبودند فرصت درخشش را از آنها گرفت، کسانی که حتی شانس شنیده شدن را نداشتند ، کسانی که زور تلاششان به شانس و سرنوشتی که تصمیمش را برای آنها گرفته بود نرسید و کسانی که برای آزاد سازی و قابل دسترس بودن دانش تلاش میکنند.

با تقدیر و تشکر از

- دوستان عزیزم ADSkeptiK و FiroozTamk بابت یادداشت ها و منابعی که معرفی و در اختیار من قرار دادند، نوشتن پایان نامه بدون کمک این جوانمردان ممکن نبود.
- آقای Graham Sellers نویسنده کتاب OpenGL Superbible بابت پاسخگویی به سوالات مستقیم من از شخص ایشان.
- جمعی از اساتید دانشگاه آزاد اسلامی رودهن اللخصوص استاد امید آقا لطیفی (از اساتید عالی من در دانشگاه رودهن که پاسخگوی بسیاری از سوالات سخت و تخصصی من بودند و قبل از قابل دسترس شدن هوش مصنوعی های قابل، اولین گزینه پاسخگوی به این سوالات بودند و سر نخ هایی که در پاسخ به سوالاتم به من میدادند به ممکن شدن این پروژه کمک بسزایی کرد)، استاد نرگس مرعشی (کمک بسیاری به ما در درک کلی از شبکه ها در دروس آزمایشگاه کردند)، استاد ساسان آزاد (به بهترین و زیبا ترین نحو ممکن اقدام به توصیف و آموزش ساختمان های داده پرداختند) و استاد زهرا صادقی (که مباحث تدریس شده توسطشان بعد ها در مدل های سه بعدی مورد استفاده ما قرار گرفت)
- همه نویسندگان کتاب های منبع که بدون آنها قطعا این پروژه ممکن نمیشد.
- انتشارات طلایی (BookGolden.ir)، ایبوک تو بوک (ebook2book.ir) که دسترسی به چاپ فیزیکی و با کیفیت بسیاری از این کتاب ها را برای ما فراهم ساختند.
- سایت ebooksworld.ir به دلیل ارائه PDF کتاب های فوق العاده مفید به صورت رایگان.
- جامعه GNU و همه افرادی که به گسترش علم و قابل دسترس تر شدن آن کمک کردند و میکنند.

فهرست مطالب

7.....	چکیده
8.....	پروژه اول: پیاده سازی پردازشگر گرافیکی با OpenGL
8.....	مقدمه
10.....	فصل ۱: فراهم کردن محیط
10.....	ابزار مورد استفاده
10.....	مود های اوپن جی ال
11.....	آماده سازی Visual Studio
21.....	کد منبع برنامه
22.....	فصل 2: روند پردازش گرافیک OpenGL
22.....	روند کلی پردازش گرافیک توسط برنامه از زاویه تعاملات سیستمی
25.....	روند ترتیبی محاسبات پردازش گرافیک توسط استاندارد OpenGL
28.....	روند فعالیت هایی که در پیاده سازی برنامه استفاده شده
30.....	فصل 3: پیاده سازی برنامه
30.....	اضافه کردن فایل های سر ایند
31.....	تعریف کلاس اصلی
31.....	تابع های عضو
34.....	تعریف متغیر های عضو
35.....	فایل های شیر و ورودی
37.....	فصل 4: نتیجه گیری و پیشنهادات
	پروژه دوم: طراحی و استقرار یک صفحه وب رسیانسیو در اینترنت بدون استفاده از کتابخانه ها (فرانت اند)
39.....	

39.....	مقدمه
42.....	کد های برنامه
43.....	فصل 1: فراهم سازی محیط
44.....	فصل 2: نقشه ذهنی رسیانسیو
46.....	فصل 3: پیاده سازی صفحه وب رسیانسیو
46.....	کد سایت
51.....	فصل 4: نتیجه گیری و پیشنهادات
52.....	پیوست 1: مقدمه ی کامپیوتر ها و برنامه نویسی (ویژه کسانی که نمیدانند از کجا شروع کنند)
63.....	سیستم های کنترل ورژن
64.....	پیوست 2: مرور و آموزش کامل مفاهیم C/C++
64.....	مقدمه
65.....	بخش اول: روال کلی برنامه نویسی C/C++
72.....	بخش دوم: مباحث C/C++
93.....	نتیجه گیری کلی
93.....	نکاتی که وقت برای پرداختن به آنها نشد...
96.....	فهرست منابع

چکیده

در این نسخه بخصوص (نسخه اول) از این پایان نامه که به دانشگاه ارائه می‌دهم بر خلاف عنوان کوتاه علاوه بر نوشتن یک برنامه ساده پردازشگر گرافیکی (با واسط برنامه نویسی گرافیک OpenGL)، به طراحی و استقرار یک صفحه وب رسیپانسیو در اینترنت بدون استفاده از کتابخانه‌ها (فرانت اند) و سپس در بخش پیوست ها به آموزش (حداقل در قیاس با دانشگاه) نسبتاً کامل و پروژه محور زبان های برنامه نویسی C/C++ و مبانی کامپیوتر بدون اتکا به تجربه شخص در این حیطه، به شیوه ای قابل درک و به منظور پاسخگویی به سوالات مهم و آماده سازی اشخاص برای فعالیت در حوضه های مختلف خواهیم پرداخت. تنها دلیل نپرداختن کامل و جزئی تر به C/C++ و حتی پروژه های دیگر زیغ وقت در این نسخه که به دانشگاه ارائه شد میباشد.

مهم: این پایان نامه تنها ترکیبی از بخشی فوق العاده اندک از پروژه های برنامه نویسی بلند مدت ترمن و یک کتاب دایما در حال تکامل به نوشته خودم میباشد که آخرین نسخه و اضافات آن درکنار فایل این پایان نامه در مخزن گیت : <https://github.com/KliffiousBiggious/the-simple-renderer> بصورت رایگان و به منظور استفاده رایگان علاقه مندان قرار میگیرند. لذا هرگونه فروش الکترونیکی این دو سند الکترونیکی بلا استثناء بدون اجازه و خواست من بوده و عملاً کلاهبرداری به حساب می آید.

فرمت این کتاب

در فصول مربوط به پروژه ها صرفاً به بینش مهندسی و پیاده سازی ای و UML (ها) خواهیم پرداخت و در بخش پیوست ها به توضیح کامل شیوه کار زبان ها، توابع و ابزار های استفاده شده بصورت جزئی میپردازیم.

کلید واژه ها

shader - OpenGL - WinAPI - Win32 - Microsoft Visual Studio - C/C++ - ترانزیستور - Host - CSS - HTML - API - مبنایهای عددی - ویندوز - گرافیک - Git

پروژه اول: پیاده سازی پردازشگر

گرافیکی با OpenGL

مقدمه

تفاوت فلسفه محاسباتی تصاویر در برابر برنامه های عادی منجر به پیدایش قطعات سخت افزاری که به عنوان کارت های گرافیک/تراشه های گرافیک/پردازنده های گرافیکی یا به اختصار جی پی یو ها میشناسیم برای رسیدن به بازدهی بمراتب بیشتر در پردازش گرافیک شد. در حالی که پردازنده های کلی با اولویت دادن به سرعت و پردازش عملیات پیچیده، با بهره گیری از تعداد بسیار کمتری هسته قدرتمند اما بسیار هوشمند در مسیر خود به پیشرفت پرداختند، جی پی یو ها به موازات آنها و با اولویت دادن به انجام همزمان تعداد بسیار بسیار بیشتری از محاسبات خیلی ساده بصورت همزمان و با استفاده از تعداد هسته های بسیار بسیار بیشتر با هوش کمتر، جایگاه خود را یافته و به پیشرفت در آن پرداختند.

واسط های برنامه نویسی گرافیک

در نتیجه روی آوردن منطقی کامپیوتر ها به استفاده از جی پی یو ها برای پردازش های گرافیکی، سیستم عامل ها، تولید کنندگان این قطعات و برنامه نویسان برای پشتیبانی بیشتر و برنامه نویسی راحت تر تصمیم گرفتند لایه انتزاعی را تعریف کنند که امکان برنامه دادن به کارت گرافیک از درون سیستم عامل به کاربران نهایی میدهد. این لایه در قالب استاندارد ها (یا همان واسط های برنامه نویسی) پیاده سازی میشود

که به آنها واسطه های برنامه نویسی گرافیک میگوییم (از این به بعد برای راحتی به آن **GAPI** میگوییم). این واسطه ها برای انعطاف پذیری بیشتر معمولاً دارای یک زبان برنامه نویسی گرافیک میانی یا **Shading Language** هستند که توسط زبان نویسان معرفی، توسط توسعه دهندگان سیستم عامل ها برای سیستم عامل قابل شناسایی و در آخر توسط سازندگان کارت گرافیک ها بصورت اختصاصی برای کارت گرافیک ها در غالب درایور هایشان قابل پشتیبانی میشوند از معروف ترین آنها میتوان به **GLSL** برای واسطه **Open GL** و **HLSL** برای **Direct 3D** را نام برد. در نتیجه زمانی که طبق راهنمایی سیستم عامل به آن **GAPI** وصل شویم، قابلیت بکارگیری توابع آن و برنامه دادن به کارت گرافیکی که از آن پشتیبانی میکند توسط کد های نوشته شده در این زبان ها یا همان **Shader** ها برای ما محیا میشود. **Direct3D**، **Open GL** و **Vulkan** همه نمونه هایی متفاوت از **GAPI** ها هستند که با وجود تفاوت هایی در نوع پیاده سازی همگی نهایتاً از روال اصلی که بالا گفته شد استفاده میکنند.

وصل شدن به GAPI ها

فرایند وصل شدن به **GAPI** ها که به آن مرحله **تعیین مفهوم** میگویند معمولاً به طور کلی با تعیین آن **GAPI** (که خود در قالب یک کتابخانه در سیستم عامل پیاده سازی میشود) به عنوان یک وابستگی و استفاده از آن در کنار **API** های مربوط به سیستم پنجره سیستم عامل ها شروع میشود. روال کلی این است که سپس با تعریف یک پنجره با پارامتر هایی خاص که در راهنمای برنامه نویسان آن سیستم عامل چگونگی تعریف آن یافت میشود، برنامه را به **GAPI** مورد نظر وصل میکنیم.

شیوه معمول: با توجه به اهمیت سرعت و نزدیکی تعامل با درایور ها و API سیستم عامل در سطوح اساسی معمولاً از زبان های کامپایلری مانند C/C++ برای نوشتن پردازشگر های گرافیکی استفاده میشود.

مرحله تعیین مفهوم میتواند خود به دو نوع کلی انجام شود:

- 1- استفاده مستقیم برنامه نویس از توابع **API** سیستم عامل مورد نظر
- 2- استفاده از کتابخانه های آماده که توابع ساده ترمبنتی بر همان توابع **API** سیستم عامل برای تعیین مفهوم ارائه میدهند.

فصل ۱: فراهم کردن محیط

ابزار مورد استفاده

در این نسخه این پایان نامه با توجه به اینکه صرفاً تمرکزمان برای نوشتن برنامه بر روی سیستم عامل های ویندوز است و به منظور سهولت و سرعت بخشیدن به فرایند نوشتن برنامه از کتابخانه های ایستا کتاب OpenGL Superbible کمک گرفتیم که بخشی از مخزن کد آن است و شیوه استفاده از آن در بخش توضیحاتش با جزئیات ذکر شده. برای استفاده از این کتابخانه و به طور کلی برنامه نویسی ویندوز از زبان C/C++، کامپایلر و برنامه Microsoft Visual Studio با پیک های ابزار Windows SDK استفاده کرده ایم که تقریباً امری الزامیست، چراکه برای بهره گیری از امکانات ویندوز نیاز به استفاده از Header های آن و کتابخانه های استاتیک (یا همان ایستا) میباشیم که ما را در زمان لینک شدن به User32.DLL متصل کنند تا استفاده از ساز و کار های ویندوز بدون ارور زمان لینک ممکن باشد. برنامه های این پایان نامه در محیط Visual Studio 2022 نوشته و اجرا شده اند.

مود های اوپن جی ال

OpenGL دارای دو مود کلی میباشد:

Core: این مود پس از تغییراتی اساسی در روند و نگرش مربوط به انجام برخی توابع در معماری های کارت گرافیک ها به وجود آمد و در آن برخی توابع قدیمی اغلب به دلایل خیلی خوب کامل منسوخ شدند و تعدادی تابع جدید و اغلب بهینه تر در آن جایگزینشان شدند.

Compatibility: بر خلاف فکر اولیه ای که احتمالاً با نام آن به ذهنتان میرسد این مود به منظور سازگاری با جی پی یو های بیشتر نیست! بلکه مودی برای سازگاری با جی پی یو های شدیداً قدیمی بسیار معدود که پشتیبانی مود Core برای آنها انجام نگرفته است، میباشد که در آن توابع منسوخ هنوز

قابل استفاده هستند اما با توجه به مهاجرت اکثر سازندگان و برنامه نویسان و متمرکز شدن آنها بر روی **Core**، تمرکز و اهمیت بسیار کمتری روی پیاده سازی **Compatibility** این **API** وجود دارد و علاوه بر آن به دلیل ساختار متفاوت جی پی یو های فعلی به طور کلی برخی از آنها قابلیت سازگار شدن با **Compatibility** را اصلاً ندارند و اجرای توابع منسوخ آن بر روی آنها میتواند منجر به ارور ها و اتفاقات غیر قابل پیش بینی شود. نتیجتاً استفاده از **Compatibility** برای برنامه هایی که تمرکزشان قابل اجرا بودن برای جی پی یو های زیاد میباشد توصیه نمیشود.

از بین این دو **Compatibility** به دلایل معلوم از **Core** استفاده خواهیم کرد. با توجه به اینکه برای راحتی کار در نسخه فعلی پایان نامه از یک **API** یا کتابخانه استاتیک (یا همان ایستا!) که فرایند ساخت پنجره آماده استفاده برای **Core** را خودکار انجام میدهد استفاده کرده ایم، ورود به جزئیات این مسئله را به روزرسانی های بعدی این پایان نامه و کتابم موکول میکنم تا به فرایند آماده سازی **Visual Studio** بپردازیم.

آماده سازی Visual Studio

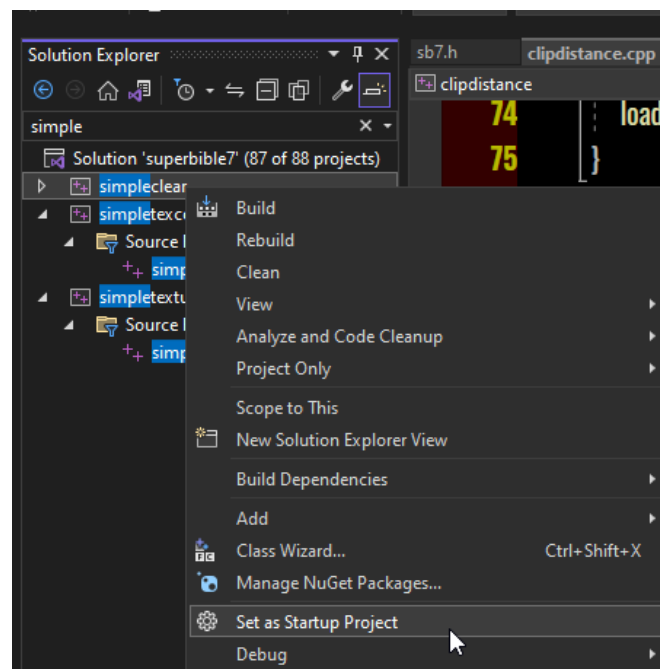
تست مخزن در Visual Studio

پس از دانلود مخزن داده شده در بالا و انجام دادن مراحل ساخت آن با **CMake** در صورت درست انجام شدن این فرایند (که ممکن است در ابتدا کمی سخت بنظر برسد)، شما باید به وضعیتی مشابه وضعیتی که در تصویر 1-1-1 میبینید برسید که در آن پروژه ها و اطلاعات مربوطه شان در آدرسی به نام **sb7code-master** بوجود آمده و همه از طریق فایل **superbible7.sln** که **solution** یا همان ابرپروژه اصلی میباشد، قابل دسترسی و نمایش هستند. برای اطمینان درست بودن اوضاع میتوانید مانند شکل 1-1-2 ابرپروژه **superbible7.sln** را در برنامه **Visual Studio** اجرا/باز نموده و اقدام به ساخت پروژه **simpleclear** بنمایید. برای اینکار ابتدا با راست کلیک کردن بر روی پروژه **simpleclear** از نوار نمایش داده شده، **set as startup project** را انتخاب کرده و سپس گزینه اجرا را بزنید. (میتوانید پس از همان راست کلیک کردن به گزینه **build** اکتفا کنید اینجا صرفاً برای محکم کاری و اجرا بلافاصله پس از ساخته شدن این راه را انتخاب کردیم) در صورتی که بدون ارور برنامه اجرا و پنجره تولید شود یعنی عملیات موفقیت آمیز بوده و کتابخانه به درستی کار میکند.

etup Files (G:) > Visual Studio Projects > Libraries > sb7code-master			
Name	Date modified	Type	Size
ssao.vcxproj.filters	10/4/2023 9:29 PM	VC++ Project Filte...	1 KB
ssao.vcxproj.user	10/4/2023 9:29 PM	Per-User Project O...	2 KB
starfield.vcxproj	10/4/2023 9:29 PM	VC++ Project	53 KB
starfield.vcxproj.filters	10/4/2023 9:29 PM	VC++ Project Filte...	1 KB
starfield.vcxproj.user	10/4/2023 9:29 PM	Per-User Project O...	2 KB
stereo.vcxproj	10/4/2023 9:29 PM	VC++ Project	53 KB
stereo.vcxproj.filters	10/4/2023 9:29 PM	VC++ Project Filte...	1 KB
stereo.vcxproj.user	10/4/2023 9:29 PM	Per-User Project O...	2 KB
subroutines.vcxproj	10/4/2023 9:29 PM	VC++ Project	53 KB
subroutines.vcxproj.filters	10/4/2023 9:29 PM	VC++ Project Filte...	1 KB
subroutines.vcxproj.user	10/4/2023 9:29 PM	Per-User Project O...	2 KB
superbible7.sln	12/21/2025 2:03 PM	Visual Studio Solu...	150 KB
tessellatedcube.vcxproj		Project	53 KB
tessellatedcube.vcxproj.filters		Project Filte...	1 KB
tessellatedcube.vcxproj.user		ser Project O...	2 KB
tessellatedgstri.vcxproj	10/4/2023 9:29 PM	VC++ Project	53 KB

Type: Visual Studio Solution
Size: 149 KB
Date modified: 12/21/2025 2:03 PM

1-1-1



1-1-2

یافتن کتابخانه در آدرس

با فرض درست ساختن ابرپروژه و زیر پروژه های Open GL Superbible توسط CMake، کتابخانه های کوچک استاتیک sb7.lib و glfw3.lib در آدرس sb7code-master/lib قرار خواهند داشت.

sb7.lib کتابخانه حقیقتاً مورد استفاده ماست که خود مبتنی به یک کتابخانه بسیار معروف و کوچک دیگر به نام glfw3.lib میباشد. GLFW کتابخانه ای چند پلتفرمی برای ساده سازی فرایند تعیین مفهوم Open GL میباشد. برای استفاده از sb7.lib، استفاده از glfw3.lib به همراه آن الزامی است.

عادت صنعتی: در نرم افزار های اختصاصی توسعه داده شده توسط شرکت های بسیار بزرگ معمولاً بجای استفاده از این کتابخانه ها مستقیماً توسط API سیستم عامل اقدام به تعیین مفهوم میکنند. کاری که پیشنهاد میشود اگر به طور هدفمند و جدی دنبال برنامه نویسی در این سطح هستید یاد بگیرید. (در نسخه های آینده این پایان نامه قطعاً به API ویندوز در بخش پیوست ها خواهیم پرداخت و میبینید که چقدر دانش از این سطح جالب و مورد استفاده است)

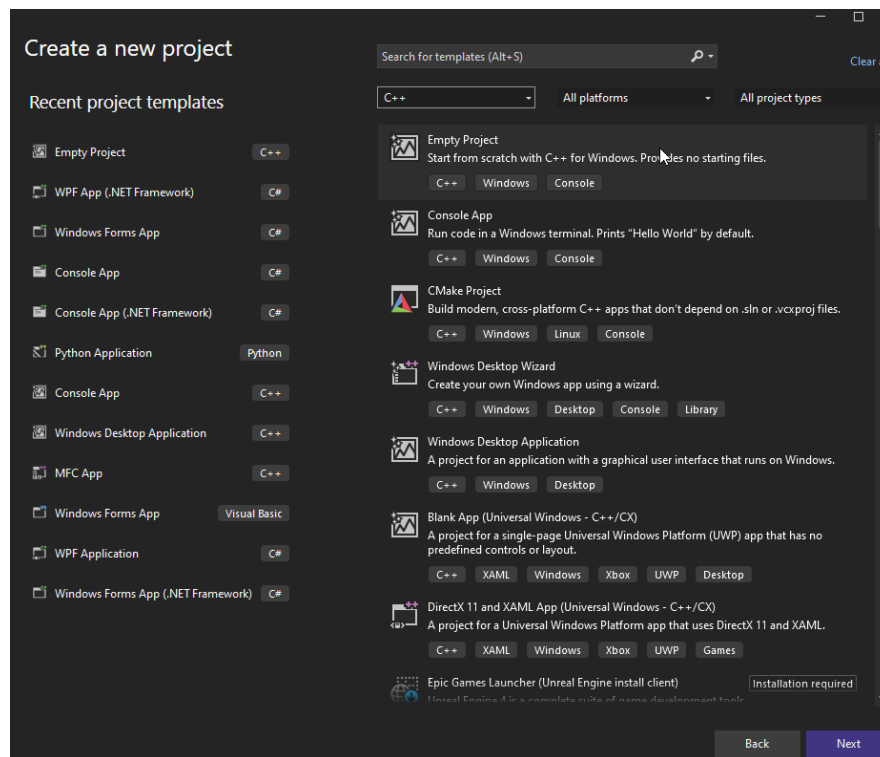
طبیعتاً استفاده از کتابخانه ها برای این است که در نهایت ما بتوانیم از توابع درون کتابخانه ها در فایل های C/C++ خود استفاده کنیم اما در حالت عادی تا قبل از لینک شدن (در زمان کامپایل شدن) فایل C/C++ خبری از توابع تعریف شده در کتابخانه ها ندارند و در نتیجه تلاش برای استفاده از آنها بر طبق قوانین C/C++، ارور استفاده از توابع تعیین نشده را به ما خواهد داد، به این دلیل کتابخانه ها معمولاً یک یا چندین فایل سرایند که نمونه یا اعلان توابع آنها در آن قرار دارد را در کنار خود ارائه میدهند تا کامپایلر تا زمان رسیدن به لینکر از کد ایراد نگیرد.

محل استقرار فایل های سرایند توابع کتابخانه sb7 در sb7code-master/include و محل استقرار سرایند توابع کتابخانه GLFW در sb7code-master/extern/glfw-3.0.4/include میباشد.

ساخت ، پیکر بندی و آماده سازی پروژه خودمان

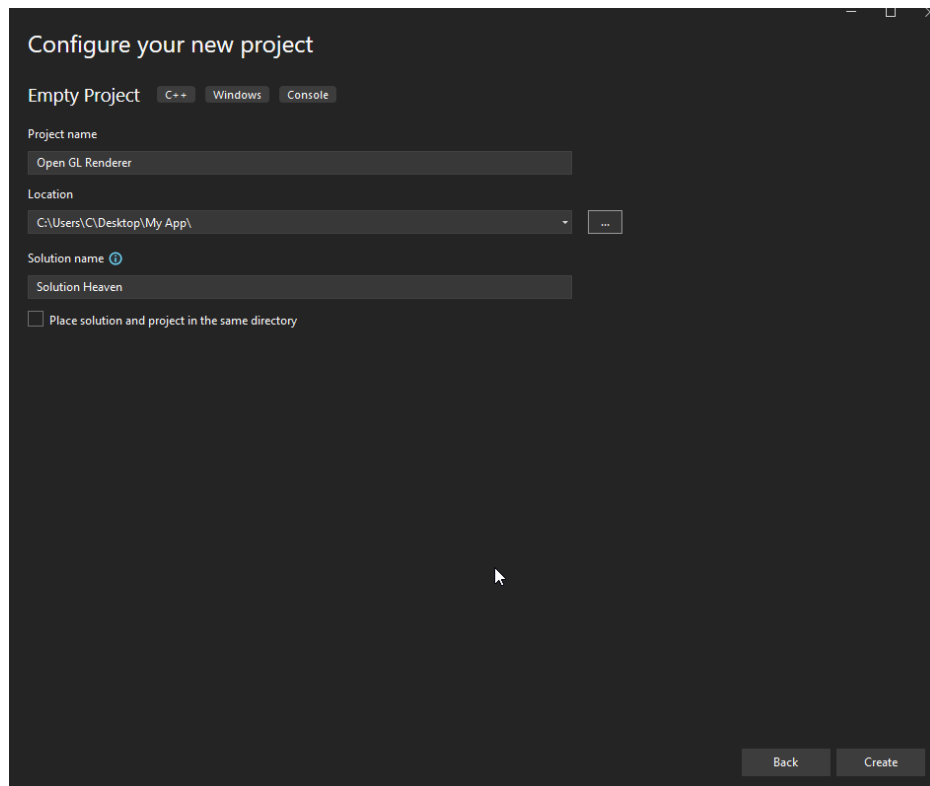
ساخت ابرپروژه

ویژوال استودیو را اجرا و از منوی اجرایی گزینه Create New Project (ساخت پروژه جدید) را میزنیم و با استفاده از فیلتر ها روی گزینه های ساختار های آماده برای برنامه نویسی C++ و برنامه نویسی Windows متمرکز شده و گزینه Empty Project را مانند شکل 1-1-3 انتخاب میکنیم.



1-1-3

سپس با کلیک بر روی next به صفحه ای مانند تصویر 1-1-4 میرسیم که از ما آدرس ساخت ابرپروژه و پروژه اولیه اش، و گزینه اینکه ابرپروژه و پروژه اولیه از یک نام استفاده کنند و فایلشان کنار هم باشد یا اینکه پروژه در یک فولدر با نام خودش در کنار فایل ابر پروژه ساخته شود را به ما میدهد.

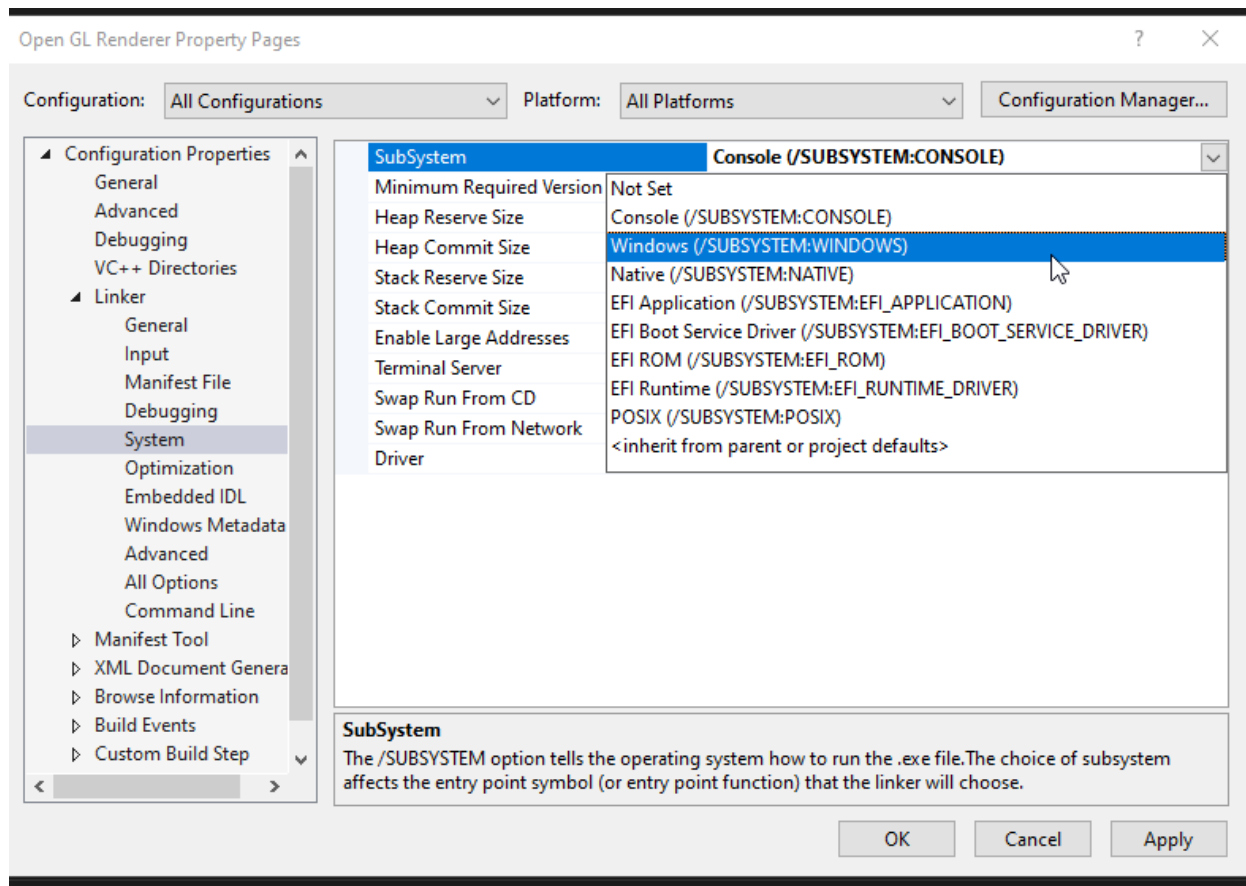


1-1-4

با کلیک بر روی گزینه Create در آدرس داده شده فولدری با نام ابرپروژه ساخته شده و درونش فایل ابرپروژه در کنار فولدری حاوی و با نام پروژه قرار خواهد گرفت.

انتخاب Subsystem مناسب

قبل از هرچیز با توجه به ماهیت پنجره محور این اپلیکیشن و استفاده کتابخانه های آن از API ویندوز باید Subsystem پروژه آنرا به Windows تغییر دهیم. برای اینکار روی پروژه راست کلیک کرده، به گزینه Properties میرویم. دقت میکنیم که Configuration روی حالت All Configurations و Platform روی حالت All Platforms باشد(اگر اینطور نبود آنها را به این فرم تبدیل میکنیم)

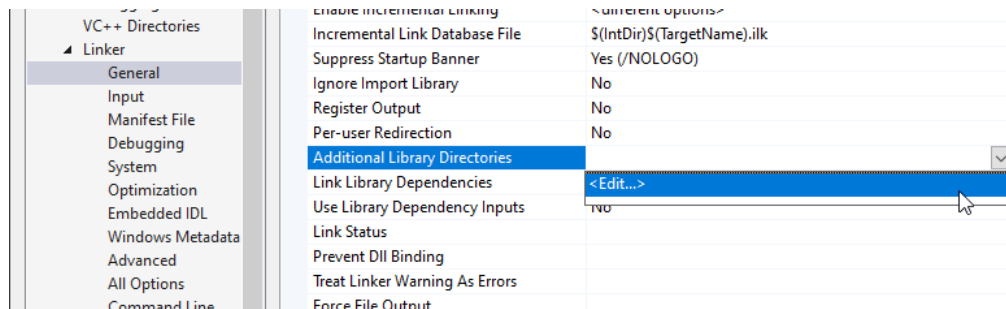


1-1-5

و سپس با رفتن به Configuration Properties → Linker → System → SubSystem
Windows مانند شکل 1-1-5 آنرا روی حالت Windows تنظیم میکنیم.

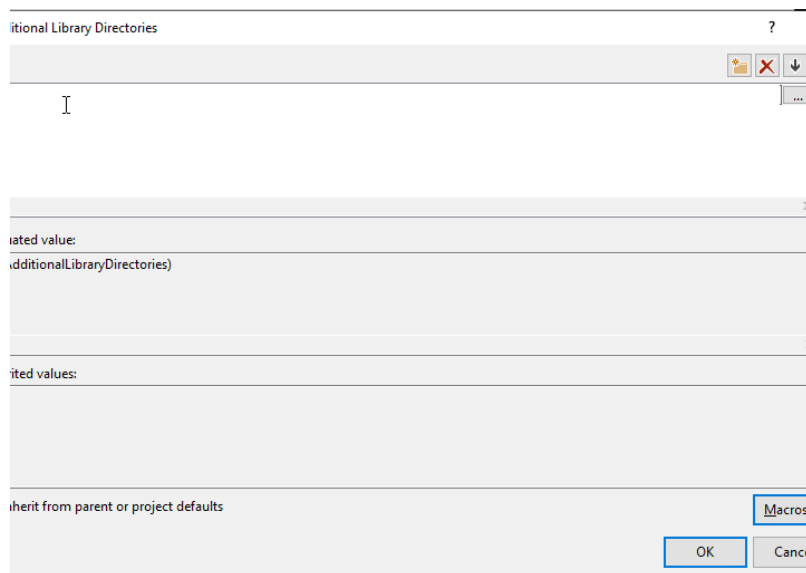
تعیین محل کتابخانه

برای تعیین محل کتابخانه مانند شکل 1-1-6 باید روی گزینه Additional Directories در تب
Configuration Properties → Linker → General کلیک کنیم. سپس با کلیک بر روی
<Edit...> پنجره ای باز میشود که به ما اجازه وارد کردن آدرس مطلق یا وابسته کتابخانه را میدهد.

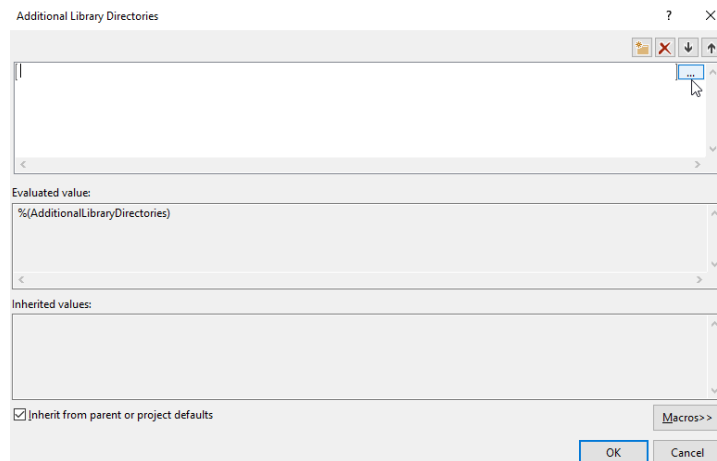


1-1-6

طبیعتاً گزینه بهتر اینست که فولدرهای `sb7code-master/include` و `sb7code-master/lib` را کپی و در کنار فولدری که فایل ابرپروژه شما در آن قرار دارد بگذارید (فایل با پسوند `.sln`) که در اینجا فولدر `Solution Heaven` میباشد، بگذارید و با استفاده از دستورات ماکرو و به شکل وابسته (مانند تصویر 1-1-10) آدرس فولدر `lib` را به عنوان یکی از محل های گشتن پروژه برای یافتن کتابخانه هایش معرفی نمایید. اما همچنان میتوانید این کار را مانند شکل 8-1-1 با کلیک بر روی ... (پس از کلیک بر روی کادر سمت چپ) و معرفی آدرس به عنوان آدرسی مطلق نیز انجام دهید. فارغ از نوع آدرس دهی، باید ابتدا با کلیک بر روی بخش سفید کادر در قسمت بالا، فیلد خالی را مشابه تصویر 1-1-7 انتخاب کنیم.

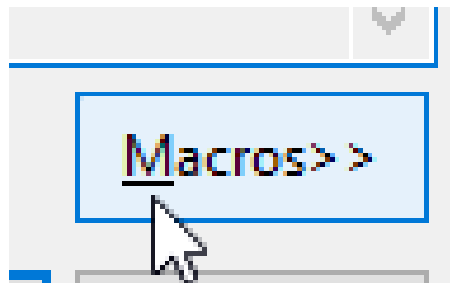


1-1-7

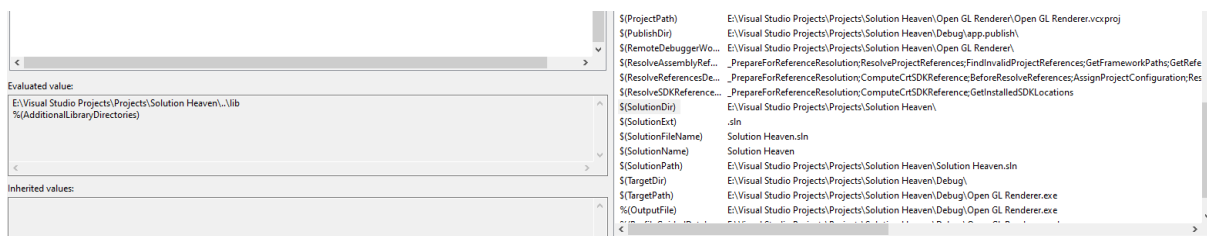


1-1-8

حال میتوانیم در همین جا با دکمه ... مسیر مطلق (فولدر که کتابخانه ها را شامل میشود) را بدهیم یا با استفاده از گزینه Macros ، دستورات وابسته را مشاهده و از آنها برای رسیدن به درون فولدر کتابخانه استفاده کنیم.



1-1-9



1-1-10

میتوان با دوبار کلیک بر روی نام ماکرو مورد نظر آنرا در فیلدی که بار آخر انتخاب کرده بودیم در جایی که اشاره گر قرار داشت اضافه کنیم و سپس با Enter فیلد را ثبت کنیم. برای فهمیدن اینکه آدرس داخل فیلد حاوی ماکرو به چه آدرسی تبدیل/ترجمه خواهد شد میتوانید بخش Evaluated Value را چک کنید و برای امتحان کردن و صحت سنجی، مقدار ترجمه شده ماکرو در بخش Evaluated value را کپی

و آنرا در بخش آدرس یک فایل اکسپلورر جایگزین و Enter بزنید. در صورت درست بودن باید محل استقرار فایل های کتابخانه ها نمایان شود. دکمه OK را میزنیم تا همه تغییرات اعمال شوند.

تعیین وابستگی های کتابخانه ای

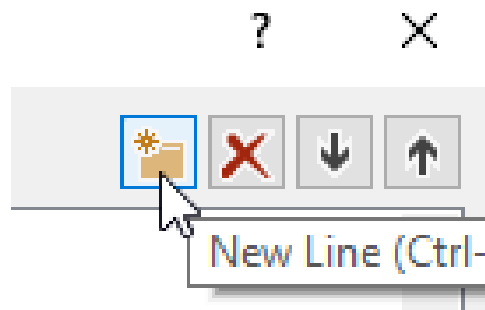
جستجو برای کتابخانه در محل های تعیین شده تا زمانی که خود کتابخانه هایی که باید برایشان جستجو شود تعیین نشده اند، ممکن نمیشود. برای تعیین آنها میبایست در مسیر

Configuration Properties → Linker → Input
Additional Dependencies مطابق بخش قبل فیلد اول را با opengl32.lib پر میکنیم (که عملاً خود پیاده سازی اوپن جی ال در ویندوز میباشد)

سپس با تغییر Configuration به Debug در پنجره

Project Properties/Project Property Pages برویم و اینبار با کلیک بر روی همان

Additional Dependencies با اضافه کردن دو فیلد بوسیله گزینه New Line در عکس 1-1-11



1-1-11

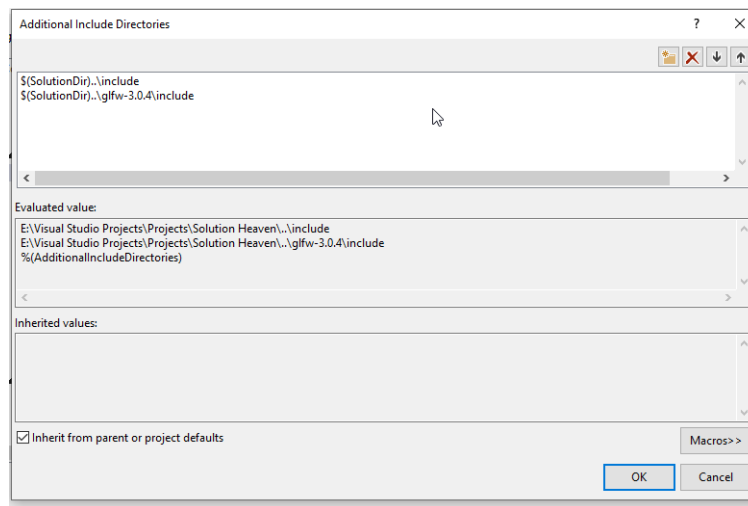
در فیلد ها اقدام به ثبت نام دو فایل glfw3_d.lib و sb7_d.lib مینماییم (بله پسوند lib. در ثبت آنها میاید) سپس با زدن Apply بخش Configuration را به Release تغییر داده و اینبار در بخش Additional Dependencies دو کتابخانه دیگر را به همان ترتیب ثبت و Apply را میزنیم.

اضافه کردن یک فایل Cpp

برای دسترسی به تنظیمات مربوط به فایل های header پروژه داشتن یک فایل C++ الزامی میباشد چرا که این تنظیمات در حالت عادی نشان داده نمیشوند! برای اینکار بر روی پروژه (نه ابر پروژه!!!) راست کلیک و به Add → New Item میرویم. از بخش سمت چپ در زبانه ها به Code → Visual C++ → Installed میرویم و با کلیک بر روی گزینه C++ File (.cpp) در بخش Name نام و در بخش Location به انتخاب به ترتیب نام و مسیر آنرا مشخص میکنیم و با زدن Add آنرا به پروژه خود اضافه مینماییم.

تعیین محل فایل های سرایند

در Project Property Pages با تنظیم Configuration بر روی All Configurations و Platform بر روی All Platforms به General → C/C++ → Configuration Properties میرویم و درست مانند بالاتر اینبار با تولید چند فیلد فرایند آدرس دهی برای Additional Include Directories را انجام میدهیم. به ترتیب آدرس سرایند های glfw و سپس آدرس سرایندهای sb7 را (طوری که هر یک در فولدر include /مربوطه خود خاتمه یابند) ثبت میکنیم که در تصویر 1-1-12 روی پروژه خود این مراحل را انجام داده ام.



1-1-12

تعیین ثابت سیستم عامل

سرایند های کتابخانه های چند پلتفرم اغلب برای گروه بندی، مدیریت و بهینه سازی نمونه/اعلان های توابعشان از ماکروهای شرطی وابسته به تعریف شدن ثابت های مخصوص سیستم استفاده میکنند. این ثابت های مخصوص نامی قرار دادی برای شناسایی و تعیین سیستم عامل ها در روند پیش پردازش میباشد. قبل از کامپایل شدن و در مرحله پیش پردازش، پیش پردازنده پروژه، تنظیمات و فایل هایش را بررسی میکند تا به یکی از این ثابت ها برسد و سپس بر اساس آن ثابت تصمیم میگیرد که فقط توابع کدام یک از سیستم عامل های مورد پشتیبانی را در سرایند تعریف و مورد دسترس برنامه نویس قرار دهد. در شرایط ما برای تعیین ثابت متناسب باید به

Preprocessor روی Project Property Pages → C/C++ → Preprocessor
Definitions کلیک میکنیم، سپس با زدن گزینه <Edit...> و اضافه کردن دو نام WIN32 و
_WINDOWS و کلیک بر روی OK و کلیک دوباره بر روی Okay پروژه را برای برنامه نویسی
ویندوز آماده کنیم.

کد منبع خود این پردازشگر

کد منبع پردازشگر گرافیک در مخزن گیت در آدرس زیر میباشد:

<https://github.com/KliffiousBiggious/the-simple-renderer>

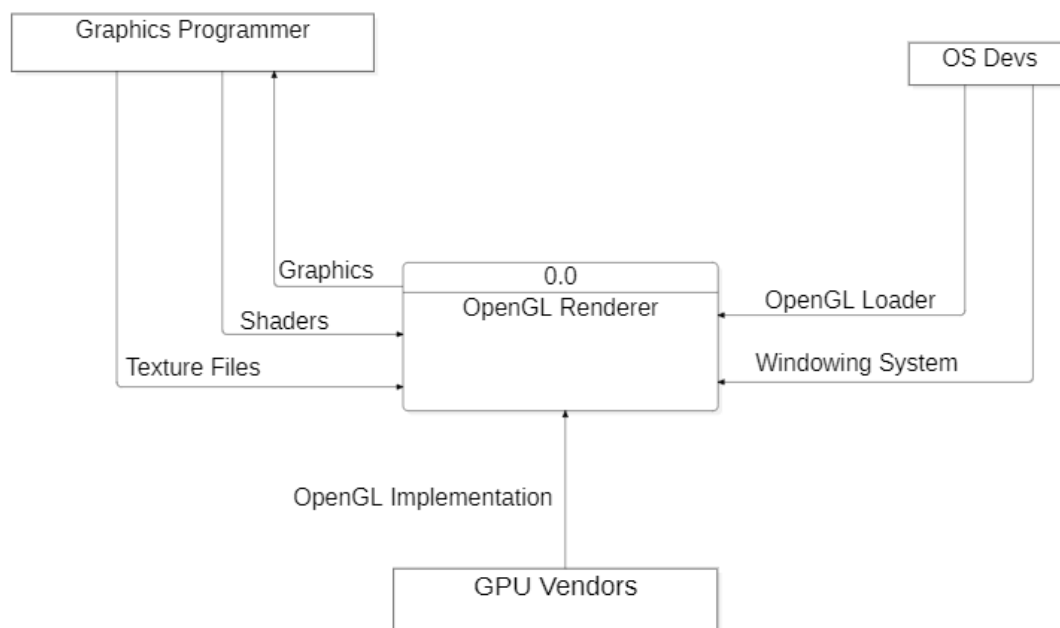
فصل 2: روند پردازش گرافیک OpenGL

روند کلی پردازش گرافیک توسط برنامه از زاویه تعاملات

سیستمی

نمودار DFD : سطح 0

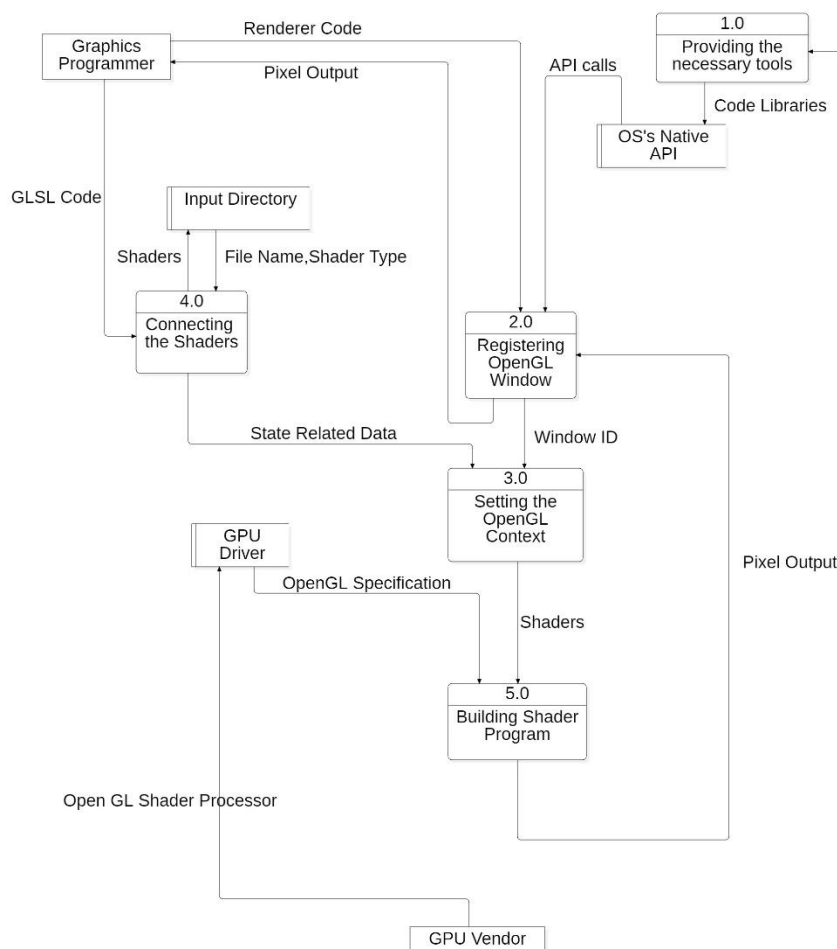
نمودار DFD سطح صفریا Context Diagram زیر نمای بسیار ساده و کلی خوبی از تعاملات رندرر OpenGL با توسعه دهندگان بخش های مورد استفاده اش توضیح میدهد.



1-2-1

در اینجا OS Dev ، Graphics Programmer ، GPU Vendors همگی موجودیت های خارجی هستند که هرکدام بخشی فرایند های لازم برای فعالیت Renderer را تامین میکنند. این روند توسط برنامه نویس گرافیک در قالب فایل های shader که در زبان GLSL (زبانی بسیار مشابه زبان C) نوشته شده اند، توسط توسعه دهنده سیستم عامل در قالب فراهم سازی سیستم پنجره و یک loader که پل بین OpenGL و درایور گرافیک میباشد و در نهایت توسط سازنده کارت گرافیک در قالب درایور های آن برای کارت گرافیک پیاده سازی میشود. خروجی این برنامه به برنامه نویس گرافیک باز میگردد.

نمودار DFD : سطح 1

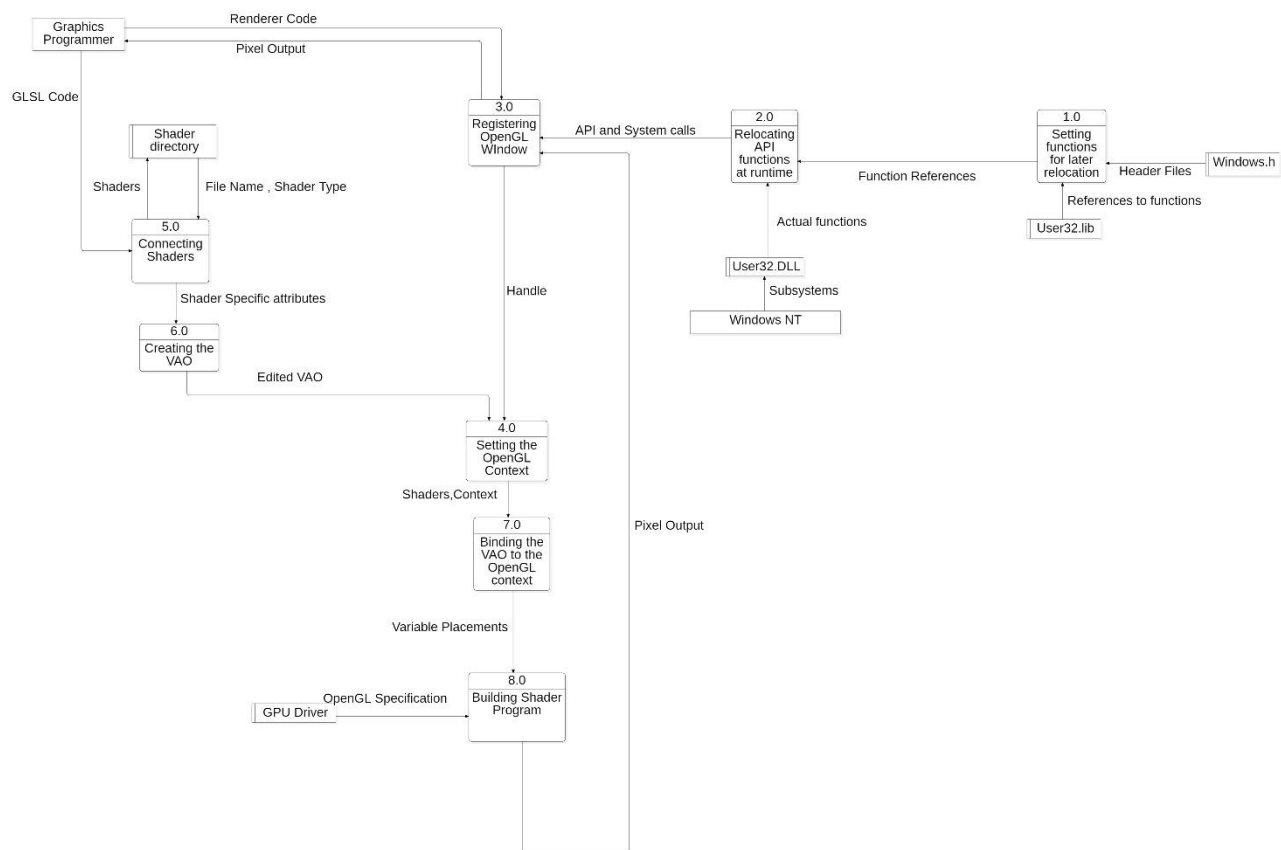


1-2-2

این نمودار صرفاً با جزئیات متوسطی از چیزهایی است که در بخش مقدمه توضیحات کافی و کامل را درمورد آنرا دادیم. پنجره نمایش گرافیک توسط تعاملات با ای پی ای سیستم عامل مورد نظر که کد آن توسط توسعه دهندگانش در قالب کتابخانه ها مورد استفاده برنامه نویسان دیگر میشود، ساخته میشود. کد های Shader همراه کدهای تعیین حالت (چرا که پنجره OpenGL عملیات ماشین حالت است) در حین تعیین مفهوم آن به آن داده شده و سپس با استفاده از درایور های جی پی یو برای آن کامپایل میشوند و در پنجره OpenGL به برنامه نویس باز میگردد.

نمودار DFD : سطح 2

این نمودار جزئیات کوچک تر تعامل زیر سیستم ها در روند کلی را در قالب اجزای دقیق استفاده شده سیستم عامل به ما نشان میدهد.



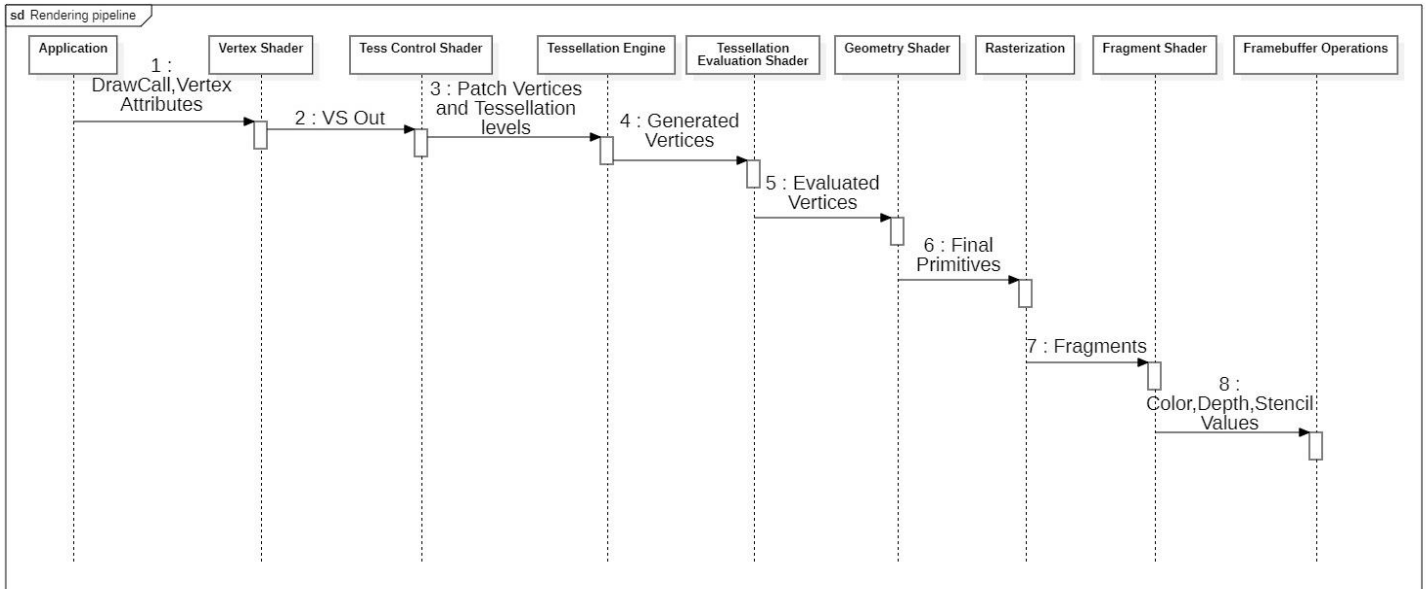
1-2-3

در این نمودار شاهد اضافه شدن مراحل ساخت و اتصال VAO ها به مفهوم OpenGL هستیم (که برای ذخیره اطلاعاتی در مورد اجرای درست شیپر ها به آنها نیاز داریم) و همچنین جزئیاتی از ارکان مورد استفاده در Windows API میباشیم. فرایند چگونگی فعالیت کتابخانه های ایستا و کتابخانه های پویا در پیوست 1 و شیوه برنامه نویسی API ویندوز در آینده در کتاب *برنامه نویسی کامپیوتر: مبانی، مفاهیم و ابزارهای مورد استفاده* به نوشته خودم در مخزن همین پایان نامه پوشش داده خواهد شد.

روند ترتیبی محاسبات پردازش گرافیک توسط استاندارد

OpenGL

پردازشی که در نمودار DFD سطح 2 نهایتاً تحت عنوان Building Shader Program خلاصه شد خود حقیقتاً خلاصه ای بسیار کوچک برای روندی که تحت عنوان Pipeline یا لوله کشی پردازش گرافیک خود OpenGL است، میباشد که در شکل 1-2-3 آنرا در قالب یک نمودار توالی نمایش داده ایم.



1-2-4

در نمودار بالا 9 خط عمر را مشاهده میکنیم که هر یک از آنها یکی از اجزای شرکت کننده در لوله کشی OpenGL میباشد (وجود برخی از آنها برای خروجی گرفتن الزامی و برخی دیگر اختیاری است):

- برنامه / Application : برنامه سی پلاس پلاس میباشد که پس از اتصال VAO با فراخوانی یک تابع رسم ورودی هایی را به مرحله بعد ارسال مینماید.

- شیدر راس/Vertex Shader : این شیدر مسئولیت انجام تعریف مختصات نقاط ابتدایی و تعریف متغیر های ورودی که قرار است به شیدر های بعدی منتقل شوند را دارد (در صفحات بعد کامل توضیح داده میشود). به دلایل مشخص یک شیدر الزامی میباشد.

- شیدر کنترل تسلیشن / Tessellation Control Shader : این شیدر با گرفتن نقاط اصلی شیدر راس به صورت گروه هایی چند تایی (به تعداد دلخواه) به ازای هر یک از این گروه ها به تعداد دلخواه تعیین شده دیگری نقاط خروجی میدهد. اختیاری است، اگر تعریف نشود شکل را به تعداد رئوس پایه نمایش میدهد.

- موتور تسلیشن / Tessellation Engine : این موتور صرفاً وظیفه انجام بخش تبدیل شکل به مدل نهاییش بعد از شیدر کنترل تسلیشن را دارد. به عنوان یه جزء همیشه وجود دارد.

- شیدر ارزیابی تسلیشن / Tessellation Evaluation Shader : این شیدر به ازای هر یک از نقاط تولیدی توسط موتور تسلیشن اجرا میشود، به همین خاطر باید حواسمان به پیچیدگی این شیدر و دفعات تکرار آن باشد. اختیاری است و با تعریف نشدنش اتفاق اضافه ای نمیوفتد.

- شیدر هندسه / Geometry Shader : به ما اجازه حذف و اضافه دستی عناصر ابتدایی (مانند خط و نقطه) را میدهد. در واقع مانند TCS بوده اما اینبار بجای اینکه تغییر توسط موتور تسلیشن انجام شود، حذف و اضافه به صورت مستقیم و دستی توسط ما انجام میشود. اختیاری است و با تعریف نشدنش همه نقاط گرفته شده از بخش های قبل بصورت خودکار از آن خارج میشوند.

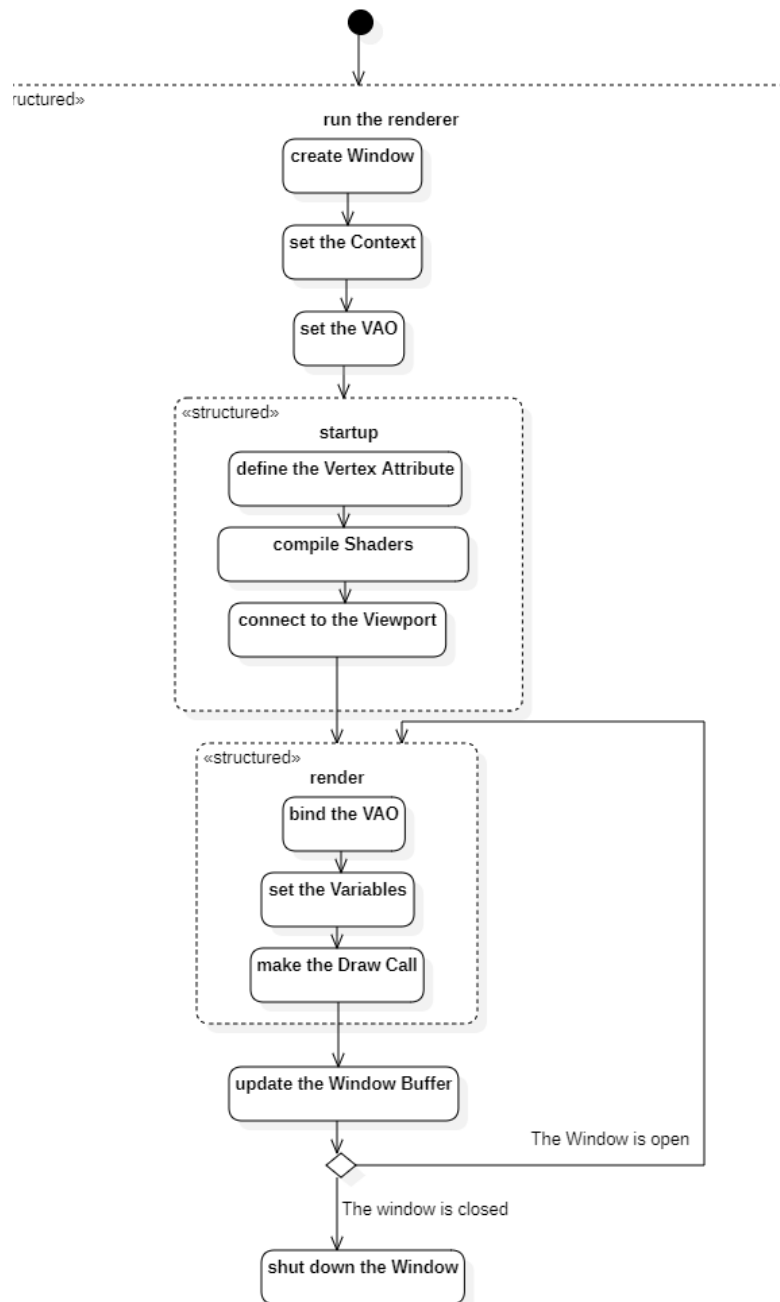
 - شطرنجی سازی / Rasterization : این عملیات همانطور که از نامش پیداست با فرایند "شطرنجی سازی" عناصر ابتدایی آنها را درون قطعه هایی در نظیر میگیرد که بعدا به عنوان پیکسل پیاده سازی میشوند.

 - شیدر قطعه / Fragment Shader : شیدر قطعه خصوصیات مربوط به قطعه از جمله رنگ آن را کنترل میکند.

 - عملیات بافر فریم / Framebuffer Operations : خروجی بخش های قبل به این بخش منتقل میشود و داخل پنجره نمایش داده میشود.
- در حال حاضر و در این نسخه از پایان نامه، در این پروژه هدف این است که برنامه ای با فضای آدرسی 64 بیتی برای ویندوز بنویسیم که در حین اجرا فایل های شیدر موجود در یک مسیر داده شده را خوانده و آنها را پردازش کند. سپس به زبان شیدر GLSL سری مندلیو را در قالب شیدر در آن پیاده سازی میکنیم.

روند فعالیت هایی که در پیاده سازی برنامه استفاده شده

در نهایت در نمودار فعالیت شکل 1-2-5 روند کلی اعمال منطقی برنامه را مشاهده میکنیم.



1-2-5

در شکل 5-2-1 میبینیم که برنامه در قالب فعالیتی ساختار یافته با تعریف متغیر های ورودی به شیدر ها ، به ساختن برنامه شیدری و سپس اتصال آنها به پنجره میپردازد و سپس تا زمانی که برنامه باز است با اتصال VAO ، مقدار دهی آن متغیر های ورودی در بخش و سپس Draw Call که یک تابع رسم میباشد، مقادیر مختلف را در قالب متغیر های ورودی برای شیدر ها فرستاده و آنها را فریم به فریم پردازش میکند. این رویه تا زمان بسته شدن پنجره ادامه دارد.

فصل 3: پیاده سازی برنامه

اضافه کردن فایل های سرایند

پیش از هر چیز باید فایل های سرایند مورد نیاز ما که اعلان توابع مورد استفاده ما در آنها تعریف شده اند را در کد دربرگیریم که عبارتند از:

- sb7.h : سرایند حاوی که اعلانات توابع کتابخانه sb7 و کلاس هایش.

- iostream : سرایند حاوی اعلانات توابع ورودی و خروجی.

- fstream : سرایند حاوی اعلانات توابع مربوط به فایل ها.

- sstream : سرایند حاوی اعلانات توابع مربوط به جریان رشته ها.

- string : سرایند مربوط به تعریف نوع رشته های کارکتری C++.

مطابق کد برنامه که در مخزن قرار داده شده است آنها را در فایل cpp. که پیش تر اضافه کردیم، با استفاده از include در برمیگیریم.

```
#include <sb7.h>
```

```
#include <iostream>
```

```
#include <fstream>
```

```
#include <sstream>
```

تعریف کلاس اصلی

در نسخه فعلی پایان نامه برنامه ابتدا در قالب یک کلاس که را دارد تعریف میشود. این کلاس که نامش دلخواه است با ارث بری عمومی از کلاس `sb7::application` حضور متغیر ها و توابع اساسی در خود را تضمین میکند. سپس این کلاس به عنوان تنها ورودی پیش پردازنده `DECLARE_MAIN` به آن داده میشود و این پیش پردازنده با ساخت یک شی از آن اقدام به اجرای تابع `run()` آن میکند که تمامی فرایندهای نمودار فعالیت صفحه قبل در آن در قالب فراخوانی های توابع دیگر انجام میشوند. در بخش زیر فرم برنامه های این کتابخانه را داریم:

```
class yourclassname : public sb7::application
```

```
{_____
```

```
_____
```

```
_____}
```

```
DECLARE_MAIN (yourclassname);
```

تابع های عضو

توابع مجازی مهم

کلاس `sb7::application` چند تابع مهم مجازی دارد که به طور پیش فرض خالی هستند و در حالت عادی فرض توسط تابع `run()` پیش فرض این کلاس فراخوانی میشوند:

- `void startup ()`: این تابع یکبار برای مقدار دهی اولیه در تابع `run` در اوایل اجرا میشود، که ما در برنامه خود با باز تعریف آن بخش های مربوط به ساختن برنامه شیدری، یافتن محل متغیر رزولوشن و زمان، ساختن آرایه `VAO` و ساختن `VBO` را در آن پیاده سازی کرده ایم و عملاً بخش های `set VAO` تا بخشی از `set variables` در آن انجام میشود. ابتدا با کمک تابع `createProgram()` به ترتیب فایل شیدر راس و سپس فایل شیدر قطعه خوانده شده و در نهایت

برنامه شیدر نهایی ساخته میشود. با فراخوانی تابع `glGetUniformLocation` محل متغیر های `u_time` و `u_resolution` در برنامه شیدر نهایی یافت و در دو متغیر `timeLoc` و `resLoc` ذخیره میشوند. آرایه `quadVertices` از نوع `float` تعریف میشود و مقادیر `x` و `y` مختصات 6 نقطه را به ترتیب طوری در نظر میگیرد که چهار راس مربع پنجره را در قالب 6 نقطه مختصاتی نشان دهد (ما مجبوریم درگاه دید `OpenGL` را که یک مربع است در قالب تعدادی مثلث که ساده ترین شکل هندسی برای پردازش است، پیاده سازی میکنیم و کمترین تعداد مثلث برای پوشش یک مربع، دو عدد میباشد، که آن دو مثلث عملاً با رسم قطر مربع مشهودند) اما چرا مختصات 6 نقطه؟! چون برای رسم هر مثلث اوپن جی ال به سه نقطه نیاز دارد نتیجتاً اگر فقط مختصات 4 نقطه باشد فقط یک مثلث را میتوان تشکیل دهد و برای مثلث بعدی دو راس کم خواهیم داشت، نتیجتاً آن دو نقطه ای که روی دو سر قطر مد نظرمان می افتند، هر یک برای بار دیگر در همان جای خود تعریف میشوند اینگونه تعداد نقطه ها به حد نصاب برای 2 مثلث رسیده اما موقعیتشان همچنان عملاً روی خودشان است و حال امکان رسم فراهم میشود. قبل از انجام رسم درگاه دید `glGenVertexArrays()` یک `VA` ساخته و به `VAO` میدهد، سپس با `glGenBuffers()` یک بافر میسازد و آنرا به `VBO` میدهد. در مرحله بعد با توابع `glBindVertexArray()` و `glBindBuffer()` ، `VAO` و `VBO` به مفهوم `OpenGL` متصل میشوند . حال که بافر به مفهوم متصل است با استفاده از تابع `glBufferData()` اطلاعات آرایه `quadVertices` درون آن ریخته میشوند و از آنجا که قار نیست تغییر کنند و قرار است برای رسم مورد استفاده قرار گیرند از `GL_STATIC_DRAW` به عنوان آخر آرگومان آخر این تابع استفاده شده. در نهایت از تابع `glVertexAttribPointer` استفاده میکنیم تا تعیین کنیم که مقادیر متغیر محل 0 در شیدر از کجا تغذیه میشود، چندین بخش از آن باید برای هر نقطه وارد شیدر شده آپدیت شوند، نوع آن متغیر ها چه میباشد، آیا نیاز به عادی سازی دارند(ندارند در اینجا چون مقادیر اعشاریند)، آیا بینشان فاصله های معین چند بایتی است (که در اینجا نیست و تعداد بایت بینشان صفر است) و در آخر بخشی از بافر که آغاز بخش متغیر ها میباشد. سرانجام با استفاده از تابع `glEnableVertexAttribArray()` این آرایه `Vertex Attribute` را برای محل 0 متغیر های شیدر فعال میکنیم.

- `void render()` : این تابع عملاً تا زمان باز بودن پنجره مدام برای هر لحظه در حال تکرار می باشد. ابتدا طول و عرض پنجره را گرفته و در دو متغیر `width` و `height` ذخیره می نماید و با `glViewport()` مختصات گوشه چپی پایینی درگاه دید را مشخص می کند. `glClear()` بافر های رنگ و عمق را پاک کرده و `glUseProgram` به برنامه می گوید که از برنامه شیدر نهایی که در `startup` تولید شد برای پردازش استفاده شود. `glUniform1f` و `glUniform2f` مقادیر `width` و `height` را به `u_resolution` و مقدار `currentTime` را به متغیر `u_time` در شیدر راس می دهند، و فراخوانی رسم شکل فعلی در قالب `glDrawArrays()` صوت می گیرد که دیدگاه OpenGL را در قالب دو مثلث که بالاتر در `startup` تعریف شدند می سازد.
- `void shutdown()` : در حین بسته شدن برنامه حافظه مربوط به برنامه را با پاک کردن اجزای آن پاک می کند.

تابع های خواندن و پردازش شیدر

- `GLuint createProgram()` : این تابع به ترتیب یک فایل شیدر راس و یک فایل شیدر قطعه را در قالب آدرس نسبی دریافت و سپس با استفاده از تابع `loadShader()` در قلب `compileShader()` اقدام به ساخت شی های آنها می کند. سپس با استفاده از `glCreateProgram()` شی برنامه شیدر نهایی را آماده ساخته و به متغیر `program` می دهد. پس از لینک کردن شی های دو شیدر راس و قطعه به آن با استفاده از `glAttachShader()` اقدام به لینک کردن آنها و ساخت برنامه شیدر نهایی می کند و بعد از چک کردن موفقیت آمیز بودن فرایند لینک شدن، فایل های کامپایل شده شیدر ها را با `glDeleteShader()` پاک می کند (چرا که در برنامه شیدر نهایی لینک شده و قرار دارند و دیگر نیازی به آنها نیست).

- `GLuint compileShader()` : به ترتیب نوع شیدر و کد شیدر را در قالب اسم یک ثابت عددی (`enum`) و یک رشته کارکتری (نوع `C++`) میگیرد. یک متغیر برای در برگیری شی شیدر ساخته و با دادن نوع شیدر به `glCreateShader()`، شی شیدر از نوع را در قالب خروجی تابع، به متغیر گفته شده میدهد. بعد یک آرایه استایل `C` درست کرده و آنرا با خروجی تابع عضو `c_str()` شی آرگومان دوم مقدار دهی میکند (اینطوری رشته کارکتری نوع `C++` ای تبدیل به نوع `C` میشود و حال قابلیت پردازش آن مهیا میشود). در نهایت با استفاده از `glCompileShader()` شی شیدر داخل متغیر کامپایل میشود، موفقیت آمیز بودن آن چک شده و سپس در قالب خروجی تابع بازگردانی میشود.

- `std::string loadShaderFile()` : آدرس یک فایل را گرفته و پس از باز کردن آن محتوای داخل آنرا با استفاده از تابع عضو `rddbuf()` به یک بافر میدهد و سپس بافر را با عضو `str` تبدیل به یک رشته از نوع `C++` کرده و آن رشته را خروجی میدهد.

تعریف متغیر های عضو

نوع متغیر های ذخیره کننده محاسبات `OpenGL` ، بصورت قراردادی اکثرا از فرمول `GL+C Type` یا نام نوع در زبان سی `GL+` پیروی میکند (`GLuint, GLchar, GLint`) نمونه هایی از این انواع هستند). برای ذخیره داشتن نتایجی مانند شیدر های کامپایل شده یا برنامه شیدری کامپایل شده نهایی یا به قول `OpenGL` "شی های `OpenGL`" از نوع `GLuint` استفاده میشود.

ما در کلاس خود (`Mandelbrot_app`) 5 متغیر عضو از نوع `GLuint` یا همان شی `OpenGL` داریم:

- `shaderProgram` : برای ذخیره شی برنامه شیدری کامپایل شده نهایی از آن استفاده میشود.
- `VAO` : که برای ذخیره کردن چگونگی پردازش ورودی های شیدر استفاده میشود.
- `VBO` : برای ذخیره بافر رئوس.

- resLoc : محل متغیر رزولوشن در برنامه شیدر نهایی.
- timeLoc : محل متغیر زمان در برنامه شیدر نهایی.

فایل های شیدر ورودی

شناسایی فایل های اجرایی برنامه به دلیل ذات توابع آدرس گیری و آدرس دهی نسبی، در زمان اجرا اعمال شده و آدرس نسبیشان با توجه به محل فایل اجرایی آنان معنا پیدا میکند. آدرسی که این برنامه برای فایل های شیدرش جستجو میکند فولدری با همین نام و در کنار فایل اجرایی برنامه میباشد و فایل هایی که به عنوان ورودی میپذیرد با نام های vertexShaderInput.glsl و fragmentShaderInput.glsl هستند که در اینجا glsl. پسوند است نه بخشی از نام.

فایل vertexShaderInput.glsl

این فایل پس از تعیین نسخه OpenGL مورد استفاده یک متغیر ورودی در محل 0 با نام position تعیین میکند که در گروهی دوتایی، مقدار ها را درون خود ذخیره میکند و آنها را در هر بار اجرا برای هر نقطه (تا سقف 6 نقطه) از بافر میگیرد (که باز خود بافر آنها را از quadVertices در تابع startup() گرفته بود) سپس در تابع main برنامه ای که باید برای هر یک از نقاط glDrawArrays() (که در render از آن استفاده شد) اجرا شود، تعریف میشود. این برنامه در هر یک از این 6 بار اجرای موازی اش، یک گروه دوتایی متفاوت را به عنوان بخش x و y مختصات نقطه ای که برایش در حال اجرا شدن است در نظر گرفته و z,w را همواره 0.0 و 1.0 در نظر میگیرد. و در نتیجه شش نقطه با مختصات متفاوت در قالب یک مربع به درستی تعیین میشوند.

فایل fragmentShaderInput.glsl

در این فایل یک متغیر خروجی FragColor چهار بخشی و دو متغیر uniform یکی دو بخشی و یکی عدد صحیح تعریف شده اند. یونیفرم های متغیر هایی هستند که مستقیماً به محلشان در شیدر با استفاده از نام وصل میشوند. که این کار قبل تر توسط تابع startup() ابتدا با یافتن و ذخیره محل این دو uniform در resLoc و timeLoc و در ادامه run() و با استفاده از glUniform2f() و glUniform1f() مقدار

دهی به آنان انجام شد. در ادامه به دلیل پیچیدگی محاسبات صرفا از کدی آماده برای پیاده سازی مندلیرو استفاده شده است و بخاطر شرایط اینترنت این کشور خیلی مقتدر و مستقل فعلا امکان یافتن جزئیات کافی برای پرداختن به آن در این پایان نامه وجود ندارد.

فصل 4: نتیجه گیری و پیشنهادات

بخشی از اقیانوسی وسیع

برنامه ای که نوشتیم صرفا استفاده از دو بخش از تعداد قابل توجه بخش های دیگر OpenGL و با استفاده فوق العاده محدود از آنها پیاده سازی این سری را در فضای دو بعدی ممکن ساخت. این برنامه بسیار ساده فقط و فقط بخشی به کوچکی یک برکه از اقیانوس ابزار ها و استفاده هایی که OpenGL برای برنامه نویسان گرافیک فراهم کرده میباشد و اگر به دلیل کم بودن وقت، کمبود جزوات کامل، تجربه و نبود اینترنت به دلیل اقتدار و استقلال خیلی زیاد این کشور نبود، قطعاً بیشتر به این موضوعات میپرداختم، لذا در صورت دسترسی به اینترنت و با تمام کردن ترجمه و خلاصه برداری از کتاب های مرجع قطعاً به روزرسانی این پایان نامه خواهم پرداخت.

اولویت های واقع گرایانه

کتابخانه های بهینه و معروف

در این نسخه از پایان نامه ما برای سرعت بخشیدن از کتابخانه sb7 که واقعا هم ساختار خوب و ساده ای برای انجام پروژه مان ارائه داد استفاده کردیم اما اگر بدنبال یک کتابخانه تمام عیار تر و بسیار بهینه و ساده برای شروع سریع پروژه ای نسبتاً بزرگ دارید، DearImGui بهترین آنهاست که متن باز و رایگان میباشد و در ویدیو های مربوط به کد منبع لو رفته GTA VI از آن برای تعریف رابط کاربری برای تغییر و نمایش مولفه های گرافیکی استفاده شده بود. SDL نیز از دیگر نام های مطرح میباشد که آن هم (اگر درست به یاد داشته باشم!) متن باز و رایگان است اما تمرکزش کمتر خلاصه به گرافیک میشود و برای بازی های ورودی ها و خروجی ها توابعی آماده دارد و در نسخه اول بازی Dying Light که بسیار بازی بهینه ای بود از آن استفاده شده بود..

استقلال از کتابخانه ها

پیشنهاد میشود اگر برای پروژه شخصی خیلی خیلی جدی هستید ترس را کنار گذاشته و به سراغ یاد گیری API سیستم پنجره سیستم عامل مورد نظرتان بروید چرا که فارغ از بدردبخوری این دانش و در هم تنیدگی آن با مبحث برنامه نویسی گرافیک، میتواند به شکل گیری یک دانش هسته ای درباره ساز و کار پنجره ها کمک بسزایی کرده و روند یادگیری کتابخانه های چند پلتفرمه را در آینده، سرعت بخشد.

پروژه دوم: طراحی و استقرار یک صفحه وب رسیانسیو در اینترنت بدون استفاده از کتابخانه ها (فرانت اند)

مقدمه

یک صفحه وب مجموعه ای از اسناد خصوصیت دهی داده شده در سطح اینترنت است که در پاسخ به درخواست هایی باید رفتار هایی مطابق با هدف و اولویت هایش ارائه دهد. طبقاً این اسناد و قواعد به محلی برای استقرار نیازمند خواهند بود که به آن خواهیم پرداخت.

هاست و دامنه

هر صفحه وب برای استقرار در اینترنت حداقل نیازمند یک سرور یا بخشی از آن میباشد که به آن هاست گفته میشود و مستلزم یک دامنه که نام آن وبسایت در دنیای اینترنت میباشد نیز است. پسوند هر دامنه با توجه به نوع دامنه خریداری شده و ارائه دهنده هاست و شرایط آن متفاوت است. هاست های رایگان معمولاً پسوند های اضافی مانند exmaple. قبل از تعیین دامنه درجه بالا یا TLD اشان (مانند .com) دارند. سرور های هاست ها از لحاظ سطح دسترسی و استفاده دارای دو گروهبندی خیلی رایج هستند:

1- هاست های اشتراکی

2- سرور های مجازی خصوصی

هاست های اشتراکی

در این هاست ها بخشی از سرور اصلی در چهارچوبی محدود و به صورت غیر مستقیم از طریق واسطی مثل یک کنترل پنل در اختیار مشتری قرار میگیرد که اغلب فقط اجازه آپلود چند فایل و ساخت یک پایگاه داده کوچک را در اختیار افراد قرار میدهد. سطح دسترسی کاربر به سیستم فقط در زبان های برنامه نویسی مورد پشتیبانی آن کنترل پنل بوده و خبری از دسترسی مستقیم به سیستم عامل و تنظیمات سرور نیست و به این دلیل است که به این سیستم ها اغلب هاست اشتراکی میگویند و نه سرور اشتراکی، چرا که سطح دسترسی شما به سرور در این گزینه در هر صورت فقط در حد یک انبار برای کد های وبسایتتان است و در این حالت نمیتوان گفت حقیقتاً "سرور" را دارید.

سرور های مجازی خصوصی

پیش از آنکه واژه "مجازی" به دلیل حقیقتاً احمقانگی و بنظر من استفاده غیر اصولی آن برای توصیف این گونه از سرور ها توسط شخصی که تصمیم گرفت از همه صفت های بمراتب بهتر این صفت را به این سرور ها دهد گمراهتان کند، باید بگوییم در سرور های "مجازی" از لحاظ منطقی سرور مجازی از صفر تولید نمیشود! بلکه سرور اصلی اجزای سخت افزاری اش را با استفاده از ماشین های مجازی تقسیم میکند، تقسیم منابع صورت گرفته و آنچه که یک سیستم بود حال به قطعه هایی که در میتوانند مانند یک سیستم مجزا عمل کنند تقسیم و قابل استفاده است ولی واقعاً چیزی مجازی نشده! بلکه فقط تقسیم شده و چون این تقسیم با قابلیت Hypervisor که مانند یک ماشین مجازی عمل میکند، انجام شده است، متأسفانه این نام مسخره در کنار نام های قلمبه سلمبه دیگر سرور ها رواج یافته است. در این سرور ها شما دسترسی مستقیم به سیستم عامل سرور و بخش قابل توجهی از تنظیمات مربوط به پیکر بندی دارید. حد دسترسی تقریباً یک درجه با بیشترین سطح ممکن فاصله دارد قابلیت نصب نرم افزار ها و... ریش و قیچی دست خودمان است اما خب قیمت ها با توجه به قدرت آن تکه که در اختیار دارید، متفاوت میباشد.

کد های صفحه وب

زبان های مورد استفاده در کد های صفحه وب را میتوان با توجه به هدفشان به دو دسته مهم تقسیم کرد:

- 1- کد های تنظیم خصوصیات محتوا (فرانت اند)
 - 2- کد های برنامه برای محتوا، اسناد و درخواست ها به سرور (بک اند و فرانت اند)
- که در این نسخه از پایان نامه در این پروژه تمرکز تقریباً فقط بر روی مورد اول است.

کد های تنظیم خصوصیات محتوا

این بخش از کد ها که مسئولیت تنظیم مفهوم، نوع، شکل، اندازه، ریسپانسیو بودن (نشان دادن رفتار متفاوت در سیستم های مختلف با توجه به خصوصیات سیستم) برای یک عنصر (منظور از عنصر متن، عکس، آهنگ و ... است) را دارند، با ترکیب دو زبان که هر یک مسئولیت بخشی از کار را به عهده میگیرید امکان ساخت یک وبسایت با رنگ و لعاب را برای ما فراهم میسازند:

- 1- زبان نشانه گذاری HTML
 - 2- زبان استایل دهی CSS برای تنظیم اندازه، رنگ و ...
- که در بخش پیاده سازی شیوه تلفیق آنها را توضیح میدهیم.

1- زبان HTML

این زبان برای تعریف نوع و مقدار و نفس عناصر مورد استفاده قرار میگیرد و نقش کلیدی و ضروری اسکلت یک صفحه وب دارد چرا که بدون استفاده از آن یک صفحه وب عملاً هیچ چیزی نخواهد داشت. برخی از بهینه سازی های مرورگر پسند (SEO) نیز با پیروی از یک سری قوانین در حین تعریف عناصر در همین زبان، انجام میشوند. پسوند فایل های این زبان به صورت پیش فرض html. بوده و معمولاً با انکدینگ UTF-8 ذخیره میشوند اما از آنجا که کد های این زبان معمولاً در تعامل با زبان های برنامه نویسی استفاده میشوند، در بیشتر مواقع آنها را در فایل های php یا js. مربوط به زبان php و javascript نیز میابید، نتیجتاً این پسوند بیشتر زمانی استفاده میشود که زبان html تنها زبانی است که داخل فایل مورد استفاده قرار گرفته است.

2- زبان CSS

این زبان برای تعریف خصوصیات ظاهری عناصر از جمله رنگ، طول، عرض، شفافیت و ... در وضعیت های مختلف استفاده میشود. به دلیل در هم تنیدگی این زبان با HTML میتواند در هر فایلی که HTML در آن قابل استفاده است، تعریف شود اما برای نظم بیشتر معمولاً آنرا در قالب فایلی انحصاراً برای این زبان با پسوند CSS نوشته میشود.

کد های برنامه

یک وبسایت میتواند اطلاعات کاربران را در محلی (پایگاه داده) ثبت، با و با استفاده از منطقی مورد استفاده قرار دهد. به دلیل شدت اساسی بودن این دو عمل زبان هایی برای انجام هر یک بوجود آمده اند که از معروف ترین های آنها عبارتند از:

- SQL: که برای ثبت اطلاعات و خوانش و مرتب سازی پایگاه داده استفاده میشود.
- PHP: زبانی که برای برنامه دهی و کنترل و مدیریت درخواست ها استفاده میشود.
- JAVASCRIPT: زبانی که کاربردی مشابه PHP دارد.

در این نسخه از پایان نامه هنوز وارد این مباحث نشده ام چراکه با وجود دانش قابل توجه به اندازه کافی برای توضیح آن آماده نیستم.

فصل 1: فراهم سازی محیط

مرورگر

قابلیت پردازش و خواندن کد های زبان های وب javascript,HTML,CSS توسط موتور های جستجو گر عادی که در روزمره از آنها استفاده میکنیم به این معنی است که حداقل در نسخه فعلی پایان نامه نیازی به دانلود نرم افزار برای کامپایل کردن کدی نداریم و داشتن یک مرورگر ترجیحا به روز محبوب مانند Chrome،Mozilla و EDGE برای پردازش مشاهده خروجی کفایت میکند.

ویرایشگر کد

به دلیل سبکی و راحتی نیز از نرم افزار ویرایشگر کد محبوب VSCode با افزونه های Live Server و HTML CSS Support استفاده کرده ایم. این دو افزونه قابلیت دیدن وبسایت خروجی کد به صورت زنده در یک پنجره مرورگر بعد از ذخیره هر تغییر و همچنین پشتیبانی از کد HTML و CSS را فعال میکنند.

هاست

قابلیت استفاده و اتصال به سایت ارائه دهنده خدمات رایگان هاستینگ که در نظر داشتیم و برای پروژه و کار خود از آن استفاده کرده بودم به دلیل فوران سطوح بی سابقه از اقتدار و استقلال در کشور وجود ندارد،احتمالا میتوانید به جای آن با ارائه کپی شناسنامه پدر،پدربزرگ ، شوهر عمه و با پرداخت هزینه یک روغن لادن طلایی ناقابل یک دامنه با پسوندی مثل eghtedar.ir. را برای یک ماه تهیه و به یک سرویس غیر مشابه داخلی دسترسی داشته باشید.

فصل 2: نقشه ذهنی رسیانسیو

نظم فایل ها

صفحه وب ما یک فایل html خواهد داشت که در کنار فولدری با نام assets میباید که جزئیات المان هایش را بر اساس نوع جزئیات در فولدر هایی با نام های توصیف کننده، دربردارد.

مقدار دهی صفر اولیه

بسیاری از المان های HTML به طور پیش فرض یک سری خصوصیات اندازه ای و ذاتی و فاصله درونی ای دارند که میخواهیم تا خودمان آنها را تعیین نکرده ایم اصلا مقدار خاصی نداشته باشند و اولین گام ما از بین بردن این مقادیر با صفر کردن فاصله ها با استفاده از * میباید که یک سلکتور به معنی "همه عناصر" میباید. مهم است که این کار قبل از همه تعاریفات دیگر انجام شود.

تعریف Grid

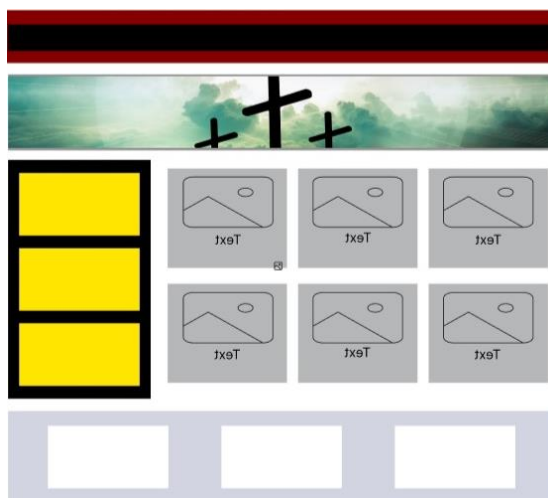
برای راحتی در برخورد و جایگذاری منطقی منظم عناصر واکنش گرا و مدیریت راحت میزان فضای اشغالی یک دورپیچ یا رپر توسط طراحان نوشته میشود که به آن مرسوم Grid میگویند. ایده فکری کلی و قراردادی Grid چهار نکته مهم اما خیلی ساده است:

- 1- تعیین ردیف ها: تعیین یک کلاس که با عناصر داخلش مانند یک ردیف رفتار شود و در صورت وجود جای فضای کافی فقط عناصر داخل ردیف بتوانند بصورت منطقی در کنار هم و در یک ردیف قرار بگیرند و عناصر خارج همواره در پایین آن بیافتند.
- 2- استفاده از مقادیر نسبی به مقدار کل قابل استفاده در عنصر والد: طول و عرض عناصر برحسب میزانی نسبی از طول و عرض کلی ای که عنصر والد در دسترسشان قرار داده و به شکل درصدی از آن تعیین میشوند. با تقسیم طول و عرض یک عنصر (معمولا 12 یا 10) اجزایی برابر

- 3- به یک واحد اندازه گیری میانی در قالب ستون ها و سطر ها می‌رسیم. با استفاده از کلاس ها یک ساختار کلی مشابه برای تعیین این که هر چند تعداد ستون باید چه درصدی از طول کلی عنصر والد را پر کنند هسته ساختار اندازه دهی رسپانسیو را بنا می‌کنیم.
- 4- تعریف شرطی و ترتیبی استفاده از واحد اندازه گیری میانی: به طور شرطی واحد اندازه گیری میانی را برای بازه های مختلف تعریف می‌کنیم، تا در هر بازه المان به اندازه رسپانسیو در نظر گرفته شده برای آن بازه فضا اشغال کند.
- 5- تعیین یک جهت برای شناوری: یک جهت که عناصر به سمت آن شناور یا فشار داده شوند تا فضاهای خالی از آن جهت شروع به پر شدن با عنصر ابتدایی کند.
- 6- تعیین فاصله رسپانسیو: عنصر به اندازه چه میزان نسبی از طرف شناوری اش فاصله بگیرد. (میدانم به احتمال زیاد متوجه نشدید اما در بخش پیاده سازی همه سوالات برطرف میشود چرا که با جزئیات بیشتری میبینید دقیقا چه میشود)

پروتوتایپ

نمونه یا اصطلاحا پروتوتایپ وبسایت به شکل زیر است. (بله میدانم چندان جالب نیست ولی در حال حاضر که دارم این نسخه از این پایان نامه را ارائه میدهم گذشته از کیفیت عالی این پایان نامه در پوشش دادن مباحث و نکاتی که حاصل یادگیری به شیوه تجربی و سخت هستند، اوضاع هیچ چیز جالب نیست پس لطفا سخت نگیرید).



2-2-1.

فصل 3: پیاده سازی صفحه وب ریسپانسیو

کد سایت

کد های مربوط به سایت در آدرس مخزن:

<https://github.com/KliffiousBiggious/Responsive-Template>

برای استفاده عموم قرار گرفته اند.

آماده سازی index.html برای شروع

فایل index

این فایل فایل اصلی میباشد که بصورت پیش فرض در سرور ها در محل مربوط به بارگذاری فایل های وب، خوانده میشود عملاً مانند نقطه شروع در برنامه ها میباشد و حکمی که تابع main برای برنامه های عادی C را دارد را برای کد های وبسایت ها دارد، فرقی نمیکند که این فایل index فایلی حاوی برنامه با پسوند php باشد یا صرفاً حاوی سند و با پسوند html در هر صورت سیستم به طور پیش فرض به دنبال فایلی با نام index برای شروع پردازش میگردد و ما با باز کردن VS Code در یک محل از پیش تعیین شده یا ساخت چنین محلی در خود ویرایشگر این فایل را در آن میسازیم.

تعریف چهارچوب html

فایل index.html را باز کرده و با تایپ کردن ! و سپس بلافاصله فشردن tab اسکلت اصلی html تایپ میشود. در تگ title اسم دلخواهی برای صفحه را جایگزین Document میکنیم. با راست کلیک بر روی پنجره کد html و کلیک بر روی گزینه Open with Live Server پنجره مرورگر پیش فرض به گونه ای باز شده که خروجی فایل پس از آخرین سیو را به صورت خودکار نشان میدهد.

تعیین محل فایل های استایل

فولدری به نام assets در کنار فایل index تولید کرده و داخل آن پوشه هایی با نام های fonts،img،css و js درست میکنیم. دو فایل به نام های gridsystem.css و style.css در پوشه css آن درست میکنیم تا از یکی برای پیاده سازی سیستم Grid و از دیگری برای استایل دهی استفاده کنیم. البته به غیر از فایل style.css تعداد زیادی فایل دیگر مشهودند که با قرار گرفتن ترتیبی در فایل style بوسیله @import در css به index.html وصل خواهند شد.

تعیین درست ترتیب لود

از آنجایی که آخرین تغییر در مقادیر یک عنصر است که بر روی آنها میماند و سلسه مراتب در خوانش HTML/CSS میتواند منجر به سربار گذاری شود، از سلسله مراتبی عقلانی در روند اتصال اسناد استایل با link پیروی میکنیم که در آن سیستم Grid قبل از استایل ها لود میشود اینگونه اگر عملکرد عجیبی از سیستم Grid دیدیم، میدانیم اول style.css باید برای سربارگذاری های محتمل چک شود.

بدنه index.html

عناصری که در داخل پنجره نمایش مرورگر پردازش میشوند به عنوان بدنه html شناخته شده و در بخش محتوای تگ body قرار گرفته و تعریف میشوند. تمرکز طراحی وبسایت نهایتا بر بیشتر از سایر بخش های ساختار html روی همین بخش بدنه هست.

پیاده سازی چهارچوب ریسپانسیو و سیستم Grid

قبل از هرچیز بهتر است به شیوه پیاده سازی واکنش پذیری عناصر پردازیم تا سبب گیج شدن نشود. این چهارچوب در قالب فایلی بنام gridsystem.css در assets/css قرار دارد.

پیاده سازی ردیف ها

هدف از پیاده سازی مفهوم ردیف این است که مطمئن شویم عناصر بعدی در صورت محیا شدن فضای کافی به یک فضای خالی و حریم ردیف دیگر تجاوز نکنند. و همواره بعد از آنها قرار بگیرد و نتوانند کنار آنها قرار بگیرند. برای پیاده سازی مفهوم ردیف در گرید کلاسی با نام row. را در نظر میگیریم که عملاً نام کلاس عنصری خواهد بود که میخواهیم به عنوان کلاس در برگیرنده عناصر یک ردیف از آن استفاده کنیم. قاعده پیاده سازی این است که با استفاده از شبه عنصر/المان after: برای row المانی خالی با شیوه نمایش table که به عنوان آخرین المان در کلاس row قرار میگیرد تعریف میکنیم. در اینجا شیوه نمایش table باعث میشود خالی بودن after که توسط content از آن اطمینان حاصل شده، مانع وجود منطقی این عنصر یا یکی شدن margin هایش با عناصر بالایی و پایینی نشود و حتما حضور منطقی در حین لود شدن داشته باشد.

تعیین و محاسبه واحد نسبی col

ما در اینجا تصمیم گرفته ایم به صورت قراردادی از یک واحد اندازه گیری که حداکثر 12 حالت دارد استفاده کنیم، نتیجتاً با تقسیم 100 که درصد کل طول قابل استفاده داده شده به عنصر فرزند توسط عنصر والدش میباشد به 12، به ستون هایی با طول 8.33% رسیده ایم. یک کلاس به ازای هر حالت (از 12 حالت) که در آن تعداد متفاوتی از ستون های همزمان پر میباشند، تعریف میشود. یعنی از حالت "طول به اندازه یک ستون پر شود" تا حالت "طول به اندازه 12 ستون پر شود" را برحسب درصدی که یک تک ستون پر میکند بدست آورده ایم تا در بخش بعد هر یک از این دوازده حالت توسط یک کلاس پیاده سازی شوند.

نکته: فارغ از تعداد ستونی که کلاس پر میکند حداقل سه حرف و حداکثر کلمه اول نام همه ی کلاس هایی که مربوط به پیاده سازی واحد نسبی خودمان در بخش بعد میشود، باید خاص آنها و با هم مشترک باشند. دلیل اینکار این است که با اتکا به این سه حرف یا کلمه اول خاص با استفاده از سلکتور خصوصیت [[، برای همه ی عناصری که بخش نام کلاسشان با آنها شروع شده یکسری خصوصیات از قبیل جهت شناوری قراردادی این عناصر (که برای وبسایت فارسی معمولاً به راست و برای وبسایت انگلیسی به چپ است) را تعریف کنیم. در این پروژه من از نام col مخفف column استفاده کرده ام که سلکتور

خصوصیت آن به شکل `[class="col-"]` نوشته شده دلیل بودن - بعد از آن این است که همانطور که در بخش بعد خواهید دید آن نیز بخشی مشترک از نام کلاس بحساب میاید و صرفاً برای محکم کاری از این - مشترک نیز استفاده شده است.

تعریف شرطی واحد نسبی در بازه های متفاوت

تقسیم بندی درصدی که در بخش قبلی به آن رسیدیم (آن 12 حالت/کلاس)، در این بخش در قالب شرط یکبار برای هر گروه بندی از اندازه های صفحه که در زیر داریم تعریف میشوند (این گروه بندی انتخاب ما بوده و استانداردهای اندازه ای متفاوتی ممکن است وجود داشته باشند):

- گروه دستگاه های بسیار کوچک : این گروه برای دستگاه هایی است که طول صفحه آنها از 1 تا 599 پیکسل میباشد. حین پیاده سازی واحد نسبی و کلاس های طولی این گروه از فرمت col-xs-num که در این فرمت تعداد واحد نسبی که طول حالت (کلاس) مورد نظر پر میکند جایگزین num میشود.
- گروه دستگاه های کوچک: این گروه برای دستگاه هایی است که طول صفحه آنها بین 600 تا 767 پیکسل میباشد. فرمت کلاس های طولی آن مانند بخش قبل است فقط بجای xs از s استفاده شده.
- گروه دستگاه های متوسط: : این گروه برای دستگاه هایی است که طول صفحه آنها بین 768 تا 991 پیکسل میباشد. فرمت کلاس های طولی آن مانند بخش قبل است فقط بجای s از md استفاده شده.
- گروه دستگاه های بزرگ: این گروه برای دستگاه هایی است که طول صفحه آنها بین 992 تا 1199 پیکسل میباشد. فرمت کلاس های طولی آن مانند بخش قبل است فقط بجای md از l استفاده شده.
- گروه دستگاه های بسیار بزرگ: این گروه برای دستگاه هایی است که طول صفحه آنها 1200 پیکسل یا بیشتر میباشد. فرمت کلاس های طولی آن مانند بخش قبل است فقط بجای l از xl استفاده شده.

تعیین این شروط با استفاده از مدیا کوئری @media به ترتیب از کوچک به بزرگ برای دستگاه ها انجام میشود که در کد مشهودند. ابتدا با تعریف مقدار طول درصدی هم ارز با تعداد ستون هایی که باید در

کلاس مورد نظر (از 12 کلاس که همان 12 حالت بودند) پر شوند در قالب خصوصیت width در آن کلاس، طول های نسبی و رسیانسیو برای دستگاه های مختلف در قالب تعریف میشود. در زمان استفاده از کلاس های مربوط به طول رسیانسیو نیز باید نام این کلاس ها در بخش نام کلاس عنصر مورد نظر، دقیقاً قبل از همه کلاس های دیگر و در ابتدا قرار بگیرند و به همان ترتیبی که بالاتر گفته شد (یعنی اول تعداد ستون هایی که در صورت xs بودن دستگاه، سپس تعدادی که در صورت s بودن آن، سپس تعدادی که در صورت md بودن آن و سپس تعدادی که در صورت l بودن آن و در نهایت تعدادی که در صورت xl بودن آن باید توسط عنصر پر شوند) تعیین شوند.

نکته: وجود فاصله از قبل از تعیین نام اولین کلاس رسیانسیو یک عنصر منجر به اعمال نشدن شرط شناوری و حقیقتاً کار نکردن خاصیت رسیانسیو میشود.

به غیر از پیاده سازی حالت های مربوط به اندازه طول خود عنصر در برخی زمان ها عناصری داریم که میخواهیم بخشی از طول کامل نسبی را پرکنند اما فاصله شان نیز به صورت نسبی از بغل سمت راست یا چپ به اندازه مشخص باشد و آن فاصله هم به گونه ای رسیانسیو عمل کند. این دغدغه دقیقاً دلیل تعریف 12 کلاس offset برای هر یک از گروه بندی های بالا است، که اینبار به جای تعریف طول به تعریف فاصله عنصر در قالب margin آن نسبت به جهت شناوری که در بخش قبل تعیین کردیم، در آن پرداخته میشود. به عبارت دیگر تنها تفاوت این کلاس ها با کلاس های مربوط به تعریف طول که در بالا داشتیم یکی در فرمت نوشتاری است در بخش قبل از num- آن یک offset- اضافه شده و دیگری استفاده از مقادیر تعداد ستون در حالت مورد نظر در قالب یک margin در سمت جهت شناوری عناصر است. به دلیل کم بودن زمان و شرایط فعلی حداقل در این نسخه به جزئیات بیشتر و حقیقتاً غیر ضروری درباره این پروژه پرداخته نخواهد شد

فصل 4: نتیجه گیری و پیشنهادات

نتیجه گیری

ما توانستیم از بیس یک سیستم Grid طراحی و با مدیریت و استفاده از ابزار هایی ساده در سطوحی ساده به وب پیچی نسبتا قشنگ برسیم، به دلیل کیفیت متن توضیحات و چندی پروژه ها و شرایط حال حاضر حداقل فعلا زمان ورود به جزئیات بیشتر یا فعالیت بیشتر برای زیبا سازی بیشتر صفحه وب در دسترس نبود اما حداقل ایده ذهنی و پیاده سازی آن توضیح داده شده است که میتواند مفید باشد.

پیشنهادهات

اگر قصد ساخت وبسایت به صورت جدی تری دارید قطعا یادگیری زبان های مانند javascript و python و csharp میتواند شما را با دنیای جدیدی از فرصت ها و راهکار ها برای پیاده سازی آشنا کند درست همانطور که کتابخانه های بسیار زیادی هستند که کار طراحی را سهولت ببخشند، برای سهولت طراحی استفاده از bootstrap و jquery بشدت توصیه میشود.

پیوست 1: مقدمه ی کامپیوتر ها و برنامه نویسی (ویژه کسانی که نمیدانند از کجا شروع کنند)

مقدمه

با توجه به اینکه مفهوم و اساس برنامه نویسی به کار گیری قابلیت های سخت افزار و زیرساخت های اغلب الکترونیکی برای دیگته کردن عملیاتی هدفمند میباشد، بهترین راه تفهیم اصول، اصطلاحات و ساختار های اساسی مورد استفاده در سیستم های کامپیوتری برای برنامه نویسان کامپیوتر، پرداختن به مبانی آن در قالب روایتی بسیار خلاصه و چکیده از تاریخچه ی کامپیوتر ها است که در ادامه به ترتیب و در تیتتر های مختلف انجام میشود، توصیه میشود اگر آشنایی حدودی با مفاهیم دارید از پیاده سازی سخت افزاری شروع به خواندن کنید. هدف این پیوست این است که چهارچوبی کلی از روال کلی کارکرد و برنامه نویسی کامپیوتر ها، دغدغه ها، راه حل ها و ابزار ها در ذهنتان داشته باشید تا در آینده در مواجهه با جزئیات حیطه مورد نظر خود را نبازید.

فلسفه کامپیوتر ها

بهترین راه خبره شدن در چگونگی بکارگیری ابزار شناخت انهاست و برای شناخت باید هدف ها، راه حل ها و اولویت ها را فهمید.

سیستم های کامپیوتری

تلاش بشر برای صرفه جویی در زمان و ساده سازی فعالیت ها و فرایند های روزانه که از مهم ترین دلایل پیشرفت و تکامل ما بوده است منطقاً ما را به سمت اتوماتیک سازی و در نتیجه آن پی بردن به اهمیت و صرفه جویی در وقت با تعریف توابع و در نهایت ساخت انتزاع هایی برای سهولت این فعالیت ها سوق داد که منجر به تحقق پیوستن مدنیت و پیشرفت ما شد. فهمیدن اهمیت مدیریت و نظم باعث پیدایش مفهوم سیستم و پی بردن به محدودیت های محاسباتی بشر منجر به پیدایش سیستم های کامپیوتری شد، که در نهایت هدف از این نوع سیستم ها درست مانند همه سیستم های دیگر، بکار گیری مجموعه ای از ارگان های اختصاصی کوچک تر برای انجام محاسبات بلند به صورت هدفمند با در نظر گرفتن و دیکته کردن شرایط بوده است، با این تفاوت که در سیستم های کامپیوتری قرار است انجام حداقل بخشی از این محاسبات به جای افراد توسط عناصر انتزاعی که به نام حسابگر ها یا کامپیوتر ها انجام شوند تا فرایند محاسبات بیشتر از اختلالات مرتبط به ضعف های احساسی، بدنی و مهم تر از همه ضعف های سرعتی انسان در امان باشند. کمک کامپیوتر ها به ما با پردازش داده ها و با استفاده از ورودی ها، خروجی ها، حافظه ها، واحد های کنترل و منطق صورت میگیرد.

اهمیت شرط ، گزاره و منطق دودویی در کامپیوتر ها

طبیعتاً تولید یک دستگاه کامپیوتر یا حسابگر بدون دانستن روش های بهینه و تجربه در کامپیوتینگ یا حساب کردن ممکن نخواهد بود؛ همانطور که دانستن همین روش های بهینه در حساب کردن، بدون فهم اصول ریاضی، و دانستن اصول ریاضی نیز بدون فهم اصول منطق غیر ممکن خواهد بود! با استناد به اهمیت شرایط در محاسبه و سیستم ها که بالاتر از آن گفتیم درست مانند دنیای واقعی برای ارائه ی تعریف خیلی خوب از شرایط نیازمند یک گزاره کاملاً درست و اغلب بلند هستیم. این گزاره کاملاً درست خود از ترکیب گزاره های بسیار کوچک کاملاً درست بدست میاید که خود آنها با استفاده از شروط مبتنی بر منطق صحت و سنجش صحت دودویی مورد استنتاج قرار گرفته اند. منطق و صحت سنجی دودویی یعنی هر گزاره در نهایت زمانی که به اندازه کافی موشکافانه تجزیه شود فقط دو حالت میتواند باشد یا

کاملا درست است یا نیست (غلط است). لازم به ذکر است که صرف ندانستن دانش کافی برای تایید درستی یا غلطی یک گزاره، این اصل که بالاخره در آخر یا کاملا درست است یا غلط را منکر نمیشود. در اینکه قطعا میتوان حالات خاص تر و بیشتری از دو حالت کاملا درست یا نادرست برای گزاره ها در نظر گرفت شکی نیست اما چون چنین حالات میانی حتی در تئوری نیازمند تعریف های جدیدی خواهند بود و منجر به ابهامات حتی بیشتر میشوند ما گزاره ها را همواره در یکی از دو حالت کاملا درست یا نادرست محدود میکنیم، و فقط آنها را در یکی از این دو گروه قرار میدهم.

سطح 0 سخت افزار فیزیکی کامپیوتر

ها (ترانزیستور ها، مبنای عددی، تراشه ها و مجموعه دستورالعمل ها)

ترانزیستور ها

با اختراع ترانزیستور ها که قطعات بسیار کوچک (به خصوص در مقایسه با لامپ های خلا که جایگزینشان شدند) هستند و با بهره گیری از مواد نیمه هادی (نیمه رسانا) میتوانند به گونه ای تغییرپذیر در لحظه، هم مانند مقاومت و هم مانند تقویت کننده جریان عمل کنند، از این قابلیت آنها برای پیاده سازی کلید های وابسته به ولتاژ و به نوعی پیاده سازی مفهوم شرط بصورت الکتریکی استفاده شد. از آنجا که همانطور که در بالاتر گفتیم آسان ترین راه رسیدن به گزاره های کاملا درست ساده کردن و تفکیک آنها با استفاده از شرط های دودویی میباشد؛ در ترانزیستور ها نیز زمانی که ولتاژ اصطلاحا به حالت اشباع میرسد و ترانزیستور کاملا رسانا شده را به عنوان حالت فعال و زمانی که جریان را میبندد به عنوان خاموش-صفر در نظر میگیرند (دوباره لازم به ذکر است که امکان تعریف حالت های بیشتر میانی که بین حالت اشباع و حالت خاموش باشند طبعا وجود دارند اما علاوه بر دلایل منطقی ذکر شده در بخش قبل

به دلیل سخت تر شدن پیاده سازی آنها و تعریف آنها بر اساس محدوده های ولتاژی بین حالت اشیاء و خاموش و امکان اشتباهات بیشتر در صورت خرابی ترانزیستور یا کم و زیاد شدن ولتاژ تصمیم بر بسنده کردن به همین دو حالت گرفته شد).

بیت

به دلیل پیروی از منطق صحت سنجی دودویی (که خود مبتنی بر اصل منطق وجودی دودویی میباشد) که در آن نهایتاً دو حالت برای هر گزاره ابتدایی ممکن بود تصمیم به استفاده از یک مبنای عددی جدید که هر رقم آن صرفاً دو حالت را پوشش میداد گرفته شد. از آنجا که عدد صفر و یک در ریاضی نیز برای اصلاً تفهیم خاصیت بودن و نبودن تعریف شدند. این دو عدد را به عنوان اعضای این مبنا که به آن مبنای دودویی یا باینری میگویند در نظر میگیریم. به هر رقم این مبنا (که میتواند صفر یا یک باشد) بیت میگویند که مخفف باینری دیجیت یا رقم دودویی است.

مبناهای عددی دیگر

با توجه به اینکه هر رقم مبنای دودویی فقط دو حالت صفر و یک را میتواند پوشش دهد، تمامی حالات ممکن برای تعداد زیاد ترانزیستور های مدارات با اعداد دو دویی نیازمند همان تعداد ارقام دو دویی بود که این مسئله منجر به استفاده از مبنا های عددی بزرگتر نظیر مبنای هشت تایی یا اوکتال و شانزده تایی یا هگزادسیمال، در کنار مبنای ده دهی یا دسیمال که به طور روزمره از آن استفاده میکنیم، برای در نظر گرفتن حالت های مجموعه هایی از این ترانزیستور ها شد. که در درس مدار های منطقی به دقت به آن پرداخته میشود.

گیت های منطقی

با بکار گیری ترانزیستور ها پیاده سازی عملگر های منطقی که فرایند های پرکاربرد منطقی مانند مقایسه را برای ورودی (ها) انجام و در غالب خروجی (ها) به ما میدادند ممکن گشت، به این عملگر ها گیت های منطقی گفته میشود. که در درس های معماری کامپیوتر و مدار های منطقی به دقت به آنها پرداخته میشود.

تراشه ها

با پیشرفت حوضه های مهندسی، الکترونیک، بالا رفتن دقت ماشین آلات صنعتی قطعات کوچک متشکل از ترانزیستور ها و مواد نیمه رسانا که برای انجام اعمال و پیاده سازی روابط و گیت (دروازه) های منطقی کار آمد بودند، محبوب شدند که به آنها تراشه یا IC گفته میشود (مخفف Integrated Circuit).

پردازنده ها

تکامل IC ها در زمینه نیاز های بشر منجر به طراحی و تولید IC های اختصاصی تر و بسیار پیشرفته که صرفا برای انجام بسیار سریع محاسبات معماری شده بودند، شد که به آنها ریز پردازنده میگویند. یک ریز پردازنده خود معمولا یک IC یا تعداد بسیار کمی IC اصلی در کنار هم است که خود از مجموعه ای از IC های فوق العاده کوچک تشکیل شده. شیوه برنامه نویسی آنها قبل از محیا شدن امکانات و سطوح بالاتر که در امروزه داریم، فقط و فقط به همین سطح 0 و 1 با تغییر فیزیکی روی ROM ها و PROM هایشان صورت میگرفت. این کار با برنامه ریزی در قالب قطع و سوزاندن فیوز ها به صورت فیزیکی انجام میگرفت که به آن OTP نیز گفته میشد. به طور خلاصه برنامه های اولیه آنها در PROM با ایجاد تغییر فیزیکی دائمی از طریق پیوند های رسانا انجام میشد، نه با دستور العمل.

ریز کنترلر ها

گذر زمان در حوضه ریز پردازنده منجر به پیدایش تراشه هایی شد که بمنظور سهولت همه ی بخش های اساسی مورد نیاز برای برنامه دهی مانند فلش و حافظه اصلی به طور پیش فرض در خود آنها تعبیه شده

است، به چنین تراشه هایی ریز کنترلر میگویند. برنامه نویسی آنها نیز قبل از در دسترس شدن امکانات و سطوح بالاتر امروزه مانند ریز پردازنده ها فقط در سطح 0 انجام میشد.

معماری مجموعه دستور العمل ISA ها و مفهوم یافتن انتزاع

تفاوت های زیاد در معماری های متفاوت ریز پردازنده ها بسته به مدل و شرکت سازنده، در مواجهه با ورودی های عددی گوناگون (که به شکل فیزیکی با ترانزیستور ها و گیت های منطقی در آنها پیاده سازی شده بودند)، منجر به اهمیت یافتن بیشتر مفهوم انتزاع و شکل گیری یکی از اولین و ابتدایی ترین و جدی ترین درجات آن در سطح 0 گشت. در تعریف انتزاع به معنی نوعی استاندارد میباشد که فرایند های مورد پشتیبانی آن استاندارد و نام یا کد آنها را به صورت لیست شده مطرح نموده و سپس توسط برنامه نویسان/سازندگان یک سخت افزار یا نرم افزار در قالب لایه ای میانی بر روی آن سخت افزار یا نرم افزار به خصوص پیاده سازی شده است، تا امکانات و فرایند های تعیین شده اش را بدون نیازمند کردن کاربر نهایی به دانستن جزئیات کوچک تر اختصاصی، در اختیار قرار دهد. این که چه مقادیر ورودی (کد عمل Operation Code) هایی در چه معماری هایی منجر به انجام چه عملیاتی در ریز پردازنده میشدند در هر معماری متفاوت و منجر به پیدایش مجموعه دستور العمل های گوناگون گشت که کدهای عملیاتی یک ریزپردازنده یا (در بیشتر مواقع) گروهی از پردازنده ها را شرح میدادند. از معروف ترین این مجموعه دستور العمل ها میتوان به x86 (که نوعی معماری CISC برای دستورات پیچیده تر با محوریت قدرت محاسباتی با استفاده زیاد در پردازنده کامپیوتر های شخصی میباشد)، ARM (که نوعی معماری RISC با دستورات ساده تر برای مصرف انرژی بهینه تر و با استفاده زیاد در پردازنده تلفن های همراه میباشد) و AVR (که برای نوشتن و اجرای دستورات ساده تر معماری RISC در میکروکنترلر ها استفاده میشود) اشاره کرد. ISA را میتوان یکی از یا (از نظر من) پایین ترین و ابتدایی ترین سطوح مهم انتزاع، حداقل در سطح 0 در سیستم ها در نظر گرفت.

زبان های توصیف سخت افزار

با تولید ریز پردازنده های قوی برای توصیف ساز و کار جزئی آنها استانداردهای زبانی یا زبان های توصیفی مانند Verilog HDL, System Verilog و VHDL بوجود آمدند که از آنها برای پیاده سازی مفهومی، منطقی و پرداختن به جزئیات سیستمی بیشتر سخت افزار ها استفاده میشد.

شغل های مرتبط با سطح 0

شغل هایی نظیر الکترونیک، تعمیرات تلفن همراه و لوازم دیجیتال نظیر هارد درایو ها حداقل به صورت متمرکز تر در سطح 0 فعالیت میکنند.

سطح 1: سیستم عامل ها

سیستم عامل ها

با گذر زمان، پیچیده تر شدن و افزایش تعداد قطعات سخت افزار های در تعامل با هم در سیستم ها، مدیریت سازگاری قطعات تبدیل به تمرکز اصلی مهندسان شد. چراکه مهندسان به دلایلی چون شرایط، کاهش هزینه ها و بازدهی بیشتر خود را دائما در استفاده از قطعات ساخت شرکت های گوناگون در سخت افزار های خود میافتنند. نیاز شرکت ها و مهندسان برای سهولت فرایند هم سو سازی و مدیریت هماهنگ این قطعات نهایتا منجر به توافق شرکت های اصلی تولید کننده با هم برای پیروی از برخی استانداردهای اساسی در حین ساخت به منظور رسیدگی به این دغدغه های مهندسان سیستم شد. در نتیجه پشتیبانی سخت افزار های بیشتر از این استانداردها، ارائه ی نرم افزاری هایی که توانایی فعالیت و هم سو سازی گسترده ای از این قطعات را داشتند ممکن و ممکن تر شد که در نهایت منجر به پیدایش اولین سیستم عامل ها گردید.

سیستم عامل در کامپیوتر برنامه ای است که با بررسی سیستماتیک قطعات موجود در سخت افزار موجبات و هماهنگی استفاده از آن قطعات را مدیریت و فراهم میسازد. سیستم عامل را میتوان یکی یا (از نظر من) مهم ترین ابتدایی ترین انتزاع (های) نرم افزاری در یک کامپیوتر مطرح کرد.

سیستم عامل ها معمولا برای رسیدن به هدفشان مانند یک پل ارتباطی با یافتن یک روال کلی برای مدیریت اجزای یک کامپیوتر کد های اختصاصی قطعات را در قالب درایور ها که توسط برنامه نویسان شرکت آن قطعه خاص نوشته شده اند بکار گرفته و با استفاده از زیر سیستم ها ، تماس های سیستمی یا سیستم کال ها و ایجاد وقفه هایشان ورودی و خروجی آنها را مدیریت و آن قطعات را بکار میگیرند.

نکته: با وجود اینکه امروزه سیستم عامل ها اغلب به طور پیش فرض برنامه های اختصاصی و جزئی قابل توجهی در کنار هسته اصلی یا کرنل خود ارائه میدهند، از زاویه سطوح انتزاع این برنامه ها هم سطح با سیستم عامل نیستند چراکه این برنامه ها روی سیستم عامل اجرا میشوند و خود آن نیستند!

شغل های مرتبط با سطح 1

توسعه دهندگان سامانه های تعبیه شده و قطعات دیجیتالی خاص که دارای دانش کافی از ISA و ساز و کار قطعات اختصاصی هستند و طراحان و مهندسان سیستم و درایور ها در این سطح با پیاده سازی چهارچوب استاندارد ها و انتزاع هایی که توسط طراح و مهندسان سیستم تعیین شده، با تعامل به یکدیگر میپردازند. به دلیل سختی فراهم بودن بسیاری از امکانات در حال حاضر اکثرا فرایندهای برنامه نویسی مربوط به این سطح نیز از سطوح بالاتر انجام شوند.

سطح 2: API ها، پشتیبانی از انتزاع و تولید برنامه های اساسی

API ها

با داشتن سیستم عامل هایی که اجزای کامپیوتر را متصل و هماهنگ میکنند حال امکان بهره گیری از ساز و کار های هسته ای در قالب ترکیب های بسیار پیچیده از تعداد بسیاری زیاده قطعه سخت افزاری فراهم شده بود. داشتن این پل ارتباطی به نام سیستم عامل به معنی پتانسیلی برای سهولت بسیار بیشتر در نوشتن برنامه های اختصاصی تر بود. استفاده از این پتانسیل دقیقا نقطه ی اهمیت بعدی برای تعداد زیادی از مهندسان بود که منجر به ساخت و پر اهمیت شدن مفهوم واسطه های برنامه نویسی یا API ها شد. API ها انتزاع های نرم افزاری هستند که بوجود آمدند تا اجازه برنامه نویسی و کنترل های قطعه های سخت افزاری را با واسطه قرار دادن سیستم عامل و سپس با در اختیار قرار دادن توابع و قابلیت های قابل استفاده سخت افزار، برای ما فراهم سازند. به عبارتی API ها عملا پنجره های برنامه نویسی برای بخش ها و اهداف به خصوص هستند و میتوانند خود وابسته به چند API دیگر باشند. همه ی نرم افزار های مورد استفاده و نوشته شده ما در نهایت به نوعی از API هایی استفاده میکنند که از جزء به کل رفته و API بومی آن سیستم عامل را میسازند.

پشتیبانی از انتزاع ها

سخت بودن برنامه نویسی مجزا و خاص منظره برای هر قطعه سیستم عامل ها را به پیشرفت، منظم تر شدن و پیروی و دیکته یک سری قواعد و استاندارد ها و داشت. API های کلی تر و قدرتمندتر دیگری بر پایه جزئی تر ها نوشته شدند. در نتیجه پیشرفت API ها سیستم عامل ها از حیث طراحی و قابلیت ها با حفظ تقریبا کامل انعطاف پذیری، بمراتب یکپارچه تر نیز شدند. امکان پی ریزی قابلیت تجزیه و فهم لایه های دیگری از انتزاع و استاندارد ها برای همه قطعات در API هایشان ممکن شد.

پشتیبانی از فایل ها و فرمت ها

امکان فهم و خواندن فرمت ها و فایل ها بصورت ورودی و خروجی از گزینه هایی بود که در این سطح با آشنا سازی ساختارهایی از بیت ها برای قطعات، پی ریزی شد.

برنامه های اساسی

اسمبلر ها

با فراهم شدن امکان پشتیبانی از انتزاع ها توسط یک API بزرگ و انعطاف پذیر مرکزی، مهندسان به فکر پیاده سازی یک زبان انتزاعی برای برنامه دادن به کل سیستم افتادند. زبان های برنامه نویسی Assembly فرمت بسیار ساده و آشنایی داشتند و با کمی اضافات و تغییرات میتوانند به زبان خوبی برای برنامه دادن به API کلی سیستم عامل شوند. پیاده سازی نسخه های کلی تر از این زبان در قالب کتابخانه ها و برنامه هایی در فرمت استاندارد و قابل درک برای API های بخش های متفاوت سیستم بود که با درک از ساز و کار سیستم عامل جوری نوشته و قرار گرفته شده بودند تا در زمان درست برای قطعه درست سخت افزاری فعال شوند (کتابخانه ها همان بسته های کد برنامه هستند که در فرمت های قراردادی استفاده میشوند و بعد تر به آنها میپردازیم). به برنامه کلی ای که با استفاده از این کتابخانه ها و استفاده از فونداسیون بخش قبل قابلیت خواندن، و تبدیل زبان Assembly پردازنده به زبان قابل فهم برای API همه قطعات فراهم ساخته بود، Assembler گفته میشد. کاربران کد خود را به اسمبلر داده و خروجی تبدیل شده آنرا را در محلی قراردادی در سیستم میافتند.

مفسر ها

با پیشرفت در بخش های قبل و بهره گیری از دستاورد های آنها مهندسان محیط های پذیرا و ترجمه کننده لحظه ای دستورات را، برای تعامل راحت تر با سیستم نوشتند. این محیط ها که از نظر پیاده سازی مشابه Assembler ها اما از حیث نوع استفاده از منابع بسیار متفاوت بودند مفسر نام داشتند. مفسر ها ابتدا به منظور غیر خطی کردن استفاده از سیستم عامل و بخشیدن آزادی بیشتر به برنامه نویس در کار با API اصلی مورد استفاده قرار گرفتند. شیوه کار آنها بدین گونه بود که با توجه به زبان مورد پشتیبانی شان، ورودی ها را دستور به دستور خوانده و در لحظه تبدیل و اجرا میکردند که کاملاً در

تضاد با منطق "کد را تبدیل و تبدیل شده آنرا در جایی بگذار" در اسمبلر ها بود. از اولین استفاده های مفسر ها برای پیاده سازی رابط کاربری کلی سیستم عامل ها یا همان ترمینال بود.

کامپایلر ها

سطح دسترسی نزدیک اسمبلی به پردازنده به معنای طاقت فرسای برنامه نویسی در آن از بعد زمانی و عیب یابی بود. برنامه نویسی در این زبان مستلزم مقدار دهی مستقیم اجزای CPU از قبیل ثبات ها و ... بود (که در نسخه های بعدی پایان نام به صورت مفصل به اسمبلی میپردازیم). مهندسان بیشتر از آنکه در حال تست ساختار های پیچیده باشند خود را درگیر پیاده سازی ساختار های ابتدایی نظیر حلقه ها، توابع بسیار ابتدایی و دغدغه هایی میافتند که هدف اصلی در آن گم شده بود. این دغدغه موجب شد تا تصمیم به ساخت زبان های انتزاعی تر گرفته شود که در آن عملیات منطقی پر استفاده توسط برنامه نویسان به صورت پیش فرض و بسیار ساده تر در دسترس باشند تا در نتیجه آن تمرکز بیشتری بتواند روی بخش های پیچیده قرار گیرد. اینگونه بود که مترجم های پیچیده تر این زبان های انتزاعی تر که کامپایلر نام گرفتند با بهره گیری از زیر ساخت های بخش های قبل ابتدا برای اسمبلر و توسط اسمبلر برای سیستم ها ساخته شدند. بوجود آمدن این زبان ها که زبان C یکی از اولین آنها بود بار بسیار سنگینی از دوش برنامه نویسان برداشت. با وجود کامپایلر ها برای این زبان های انتزاعی ضرورت استفاده از زبان فوق العاده ابتدایی اسمبلی جز در مواقع خاص عملا از بین رفت. کتابخانه های نوشته شده به زبان اسمبلی نیز در قالب این زبان های انتزاعی بازسازی شدند و رفته رفته مهاجرت به این زبانها صورت گرفت.

استاندارد های زبان های برنامه نویسی

اسمبلر ها ، مفسر ها و کامپایلر ها صرفا انواعی از برنامه های مترجم بودند اما توقعات از اینکه مترجم هر زبان برنامه نویسی باید برای سیستم عاملش چه حداقل هایی را ارائه دهد نه تنها در مورد مترجم ها یکی نبود بلکه فقط هم محدود به مترجم ها نبود. برخی توابع و کتابخانه های اساسی که به طور پیش فرض در کنار یک مترجم ارائه میشدند و برنامه نویسی بر اساس آنها انجام شده بود در کنار مترجم های دیگر اصلا وجود نداشتند! در نتیجه نویسندگان زبان های برنامه نویسی دستورات نوشتاری، قواعد و توابعی که باید به عنوان بخشی از زبان (کتابخانه اصلی آن) در مترجمشان ارائه شوند را در قالب

استاندارد هایی معرفی کردند. در نتیجه با چک کردن پشتیبانی مترجم از استاندارد زبانی ای که در آن برنامه نوشتید، میتوانید که از سازگاری کد و مترجم اطمینان حاصل کنید.

IDE ها

نرم افزار هایی که با استفاده از زیر ساخت های در دسترس بخش های قبل تمام مجموعه لازم از مترجم تا برنامه ای برای نوشتن و ثبت کد را در اختیار کاربران قرار میدادند تحت این نام رفته رفته در دسترس عموم قرار گرفتند.

شغل های مرتبط با سطح 2

تقریباً 90 درصد شغل های مربوط به برنامه نویسی (شامل شغل های سطح 1) از جمله طراحی سایت، گرافیک، امنیت شبکه و ... در حال حاضر به دلیل فراهم بودن زیر ساخت های زیاد در این سطح فعالیت میکنند.

دیگر موضوعات

مجوز های کد برنامه

کد های برنامه ها تحت مجوز هایی برای تعیین نوع و چهارچوب استفاده از مجوز هایی مانند GPL و MIT و ... استفاده میکنند. در نسخه های بعدی این پایان نامه شاید به آنها بپردازیم.

سیستم های کنترل ورژن

برای کار کردن گروهی بر روی یک نرم افزار برنامه نویسان نیازمند سیستمی هستند که قابلیت کار بر روی یک کد و ثبت تاریخچه تغییرات روی آنها را داشته باشد تا در صورت اشتباهات بزرگ همه چیز از بین نرود و بتوانند به حالت قبل برگردند، این دقیقاً دغدغه ای بود که باعث تولد سیستم های کنترل ورژن یا VCS ها شد. Git یک نمونه محبوب، رایگان و متن باز آن است که به دلیل زیغ وقت به آن پرداختم اما در نسخه بعد پایان نامه حتماً به آن پرداخته خواهد شد.

پیوست 2: مرور و آموزش کامل مفاهیم

C/C++

مقدمه

C از اولین و بی شک حداقل یکی از بهترین (به نظر من بهترین) زبانهای کامپایلری بود که در روند پیشرفت حوضه کامپیوتر متولد شد. در حالیکه قاعده نوشتاری ساده و قابل درک آن و هوشمندانگی قوانین مفهومی آن پیش بینی کد اسمبلی آنرا بسیار راحت و خود این زبان را جذاب میکند، برنامه نویسی با C برای هر کسی نیست. برنامه نویسی با C قواعد محدود کننده ای دارد که برای بسیاری جالب نبوده و زمانگیر بنظر میرسد در حالیکه از طرف دیگر به دلیل نبود یک سری از سخت گیری های دیگر (نظیر چک کردن هم نوعی برخی انواع داده که بعد تر به آن میپردازیم)، این اجازه را میدهد که یک سری اشتباهات وحشتناک بدون اخطار دادن ترجمه و کامپایل شوند و در حین اجرا گریبان گیر شوند. در C++ تعداد زیادی از سخت گیری ها برای جلوگیری و شناسایی از اشتباهات فاحش به دلایل خوبی اضافه شدند و علاوه بر آن امکان برنامه نویسی شی گرا که توضیحش را میدهیم، در کنار کتابخانه های C (هم کتابخانه خود C هم نسخه های شی گرا شده آنها) به کاربران در اغلب استانداردها ارائه میشوند. با توجه به درهم تنیدگی شدید این دو زبان امکان آموزش هر دو در کنار هم فراهم و به شما ارائه داده میشود.

به دلیل زیغ وقت و زمانگیری در این نسخه آموزش نصب IDE های آنها قرار داده نمیشود و با فرض اینکه یک IDE با پشتیبانی پیش فرض C/C++ را برای سیستم عامل مورد استفاده دانلود و نصب کرده اید، به آموزش مباحث اصلی به صورت مفید برای شروع جدی برنامه نویسی پرداخته میشود.

شیوه متفاوت آموزش من

به دلیل پروژه محور بودن این آموزش تصمیم گرفتم آنرا در قالب دو بخش ارائه دهم که اولی به صورت ترتیبی و دیگری به صورت رفرنس در صورت نیاز مورد استفاده لحظه ای قرار میگیرد:

- بخش اول: صرفاً به روال کلی برنامه نویسی در این زبان برای پروژه ها و تکنیک ها و استراتژی های تجربی میپردازد.
- بخش دوم: به تدریس اصول زبان ها و سر فصل ها میپردازد.

بخش اول: روال کلی برنامه نویسی C/C++

روند کلی ساخت برنامه های C/C++ از کد

برنامه های نوشته شده در C/C++ معمولاً از دو نوع فایل ساخته شده اند:

- فایل های کد منبع: این فایل ها به فاز ترجمه ارسال میشوند. پسوند فایل های کد منبع C باید c. و پسوند فایل های کد منبع C++ باید cpp. باشد.
- فایل های سرایند: هدف وجودی این فایل ها صرفاً مورد کپی قرار گرفتن محتوایشان در فایل های سرایند دیگر و در نهایت قرار گیری در فایل های کد منبع میباشد. پسوند سرایند های C باید h. و پسوند سرایند های C++ باید h. یا hcc. باشد.

ابتدا کد های برنامه در قالب مجموعه ای از فایل های سرایند و منبع توسط IDE به فاز پیش پردازنده کامپایلر داده میشوند. پیش پردازنده فایل های منبع را باز میکند و در صورت مشاهده عملگر های پیش پردازنده به انجام آنها میپردازد (کپی شدن فایل های سرایند نیز در این فاز انجام میشود و بعد از آن دیگر کامپایلر کاری با آنها ندارد) سپس فایل های کد منبع مورد بررسی های دیگر و ترجمه قرار میگیرند (در نسخه های بعد پایان نامه به آن بررسی ها نیز میپردازیم). برای هر یک از فایل های کد منبع یک فایل که به آن فایل شی گفته میشود و به زبان قابل فهم برای سیستم عامل است تولید میشود. فایل های شی به بخشی بنام لینکر داده میشوند تا نقطه تماس و استفاده شان از توابع و متغیر های یکدیگر شناسایی و پیوند داده شود و فایل به یک فایل نهایی برنامه تبدیل شوند.

نکته: فایل های نهایی لزوما همیشه فایل های اجرایی نیستند (که در نسخه های بعد باز به آنها نیز کامل میپردازیم) اما ما فعلا با فایل های اجرایی سر و کار داریم.

مرحله 0: آماده سازی

تعیین شدن بخش ها و توابع مورد استفاده

قبل از اقدام برای شروع برنامه نویسی باید حد اقل نقشه ذهنی کلی و حدودی از توابع، کلاس ها، بخش ها و فایل هایی که میخواهیم از آنها استفاده کنیم یا آنها را بنویسیم داشته باشیم.

ساخت مجموعه ی کد ها

فارغ از IDE مرحله صفر برنامه نویسی ایجاد مجموعه یا پروژه برنامه نویسی در آن با و سپس تنظیم زبان برنامه نویسی پروژه میباشد. انجام اینکار در IDE ها از لحاظ ظاهری متفاوت است اما یک سری قواعد هستند که باید در هر صورت حتما رعایت کنید:

- قواعد پسوند نام فایل ها (در بالاتر گفتیم)
- فایل های برنامه به عنوان به عنوان بخشی از مجموعه حساب شوند (نه اینکه صرفا در فولدر آن باشند، بلکه به عنوان فایل هایی که به کامپایلر فرستاده میشوند نیز ثبت شوند)
- در صورت استفاده از کتابخانه های دیگر و ... آدرس های آنها به عنوان آدرس های قابل گشتن معرفی و تنظیم شوند.

تعیین شدن نقطه شروع برنامه

هر برنامه بالاخره باید از یک نقطه شروع شود. این نقطه شروع که آغاز نیز لقب میگیرد در C/C++ همواره در قالب یک تابع شروع کننده میباشد. لینکر با گشتن به دنبال این تابع شروع کننده در فایل های شیء آدرسی که نام را در آن میابد را ثبت میکند. این آدرس حین اجرای برنامه برای اجرای آن تابع به سیستم عامل داده میشود. به طور پیش فرض (مخصوصا در مواقعی که برای بخش ترمینال سیستم عامل برنامه مینویسید و از کتابخانه هایی جز کتابخانه استاندارد C/C++ استفاده نمیکنید) معمولا تابع شروع کننده باید به شکل یکی از توابع زیر تعریف شود:

- `int main (void)`

برای برنامه هایی که در زمان اجرا ورودی و پارامتری به آنها ارسال نمیشود. بیشتر آنرا در برنامه های صرفاً به زبان C میبینید (برای C++ کاملاً بلامانع است) و دلیلش را در مورد بعد میفهمید.

- int main ()

در برنامه های C++ بیشتر دیده میشود. دلیل حذف void این است که کامپایلر C++ آنرا به صورت خودکار به شکل گزینه قبلی درمیآورد اما کامپایلر C این گزینه را به عنوان تابعی با ورودی های تعیین نشده در نظر گرفته و وابسته به کامپایلر مورد استفاده عکس العمل های متفاوتی نشان میدهد.

- int main (int argc , char* argv[])

برای ارسال پارامتر و مقادیر به برنامه هایی که از محیط ترمینال (یا همان cmd یا command prompt) سیستم عامل آنها را اجرا میکنید و محیط ترمینال سیستم عاملشان از پارامتر ها و ورودی های C/C++ پشتیبانی میکند و قرار است مقداری از بیرون بگیرند (مانند آدرس یک یا چند فایل که با رفتن به آن عملیاتی انجام دهند و...) استفاده میشود. این تابع ورودی های داده شده را در غالب کلمه ها (کارکتر های پشت سر هم بدون فاصله) شمرده و در قالب رشته های کارکتری برای استفاده در آینده توسط برنامه تان ذخیره میکند. تعداد این کلمات در argc و خود این کلمات در آرایه ی رشته های کارکتری argv ذخیره میشوند. این دو نام انتخابی اند و عوض کردن آنها مشکلی ایجاد نمیکند ولی یک نوع قرارداد خوب است و حقیقتاً دلیلی برای عوض کردن آنها وجود ندارد.

برای علاقه مندان: این دو متغیر با استفاده از CRuntime یا به اختصار CRT مقدار دهی میشوند. CRT یک بخش کد و مرحله ابتدایی اضافه شوند به فایل اجرایی نهایی میباشد. در زمان اجرای فایل نهایی اجرایی نرم افزار از ترمینال CRT قبل از تابع شروع کننده اجرا شده و ورودی ها را در یک پشته ذخیره میکند. سپس مقدار دهی اولیه argv و argc را با استفاده از آن پشته انجام میدهد.

اما در مواقع استفاده از کتابخانه های خاص تابع شروع کننده میتواند در غالب یک پیش پردازنده پیاده سازی شده شود یا اصلاً کلاً فرق کند (که هر دوی این سناریو ها را عملاً در پروژه اول این نسخه از پایان نامه دیدیم) در چنین صورتی احتمالاً نیازمند پیکر بندی IDE مورد استفاده و کامپایلر برای شناختن آن تابع جدید به عنوان شروع کننده خواهید بود.

یکی از توابع شروع کننده اختصاصی و غیر پیش فرض اما خیلی پر استفاده دیگر

`int WINAPI WinMain( HINSTANCE hInstance, HINSTANCE hPrevInstance, LPSTR lpCmdLine, int nShowCmd )`

میباشد. آرگومان های آن ممکن است در جاهایی کمی فرق کنند اما به طور کلی این تابع شروع کننده برای برنامه هایی استفاده میشود که از بخش های جزئی و اختصاصی تر ویندوز استفاده میکنند (مانند برنامه هایی که پنجره ها گرافیکی ویندوز درست میکنند).

مرحله 1: تست اولیه

با استفاده از پیش پردازنده `#include` سرایند های مورد استفاده خود را در یکی از فایل های منبع مجموعه در برمیگیریم (اگر فایلی نداریم یک فایل جدید در آن ایجاد/اضافه میکنیم). سپس با تعریف تابع شروع متناسب با نیاز در بدنه آن از توابع آن ها استفاده میکنیم. رفتن به محل سرایند ها برای چک کردن نمونه تابع اطلاعات مفیدی درباره چستی و چگونگی پارامتر های آن به ما میدهد. با استفاده از گزینه **Build and Ruin** یا گزینه ای مشابه برنامه را برای معماری x86 یا x64 اجرا میکنیم.

ارور ها

در صورت وجود ارور با چک کردن نوع آن شروع میکنیم. اگر کلمه ای مانند `LINK` در ارور دیدیم مربوط به زمان لینک کردن و در غیر اینصورت و مشاهده :یا (`Undeclared` در ارور به احتمال قوی مربوط به فاز کامپایل میباشد.

مرحله 2: پیاده سازی ماژولار بخش های مختلف برنامه

ماژولار سازی به معنای تنظیم بخش های برنامه به گونه ای است که با ایجاد تغییرات و بروزرسانی های پیش پا افتاده در یک بخش آن بخش های دیگر مجبور به بازنگری کامل نشوند. اول با تکنیک های ماژولار سازی ریشه ای تر مربوط به C (که در C++ نیز کاملاً قابل استفاده هستند) شروع و بعد به تکنیک های قابل استفاده با ابزار اضافه شده در C++ میرویم:

تکنیک های C :

ماژولار سازی در C معمولاً در قالب جفت هایی از فایل سرایند اعلانات ماژول و فایل کد منبع ماژول پیاده سازی میشود. تعریف توابع و ساختار های مورد استفاده و مرتبط به هم هر ماژول در یک فایل کد منبع c. با نام های توصیف کننده ماژول قرار میگیرند. سپس اعلان/نمونه های این توابع و ساختار ها در فایل های سرایند h. همنام با آن فایل های کد منبع نوشته میشوند و در فایل هایی که از این توابع ماژول استفاده کرده اند با `#include` در برگرفته میشوند. اینگونه در زمان فاز ترجمه، کامپایلر ها آشنایی سطحی با اسم و خصوصیات آن خواهند داشت و ایراد نمیگیرند تا نهایتاً در فاز لینک شدن محل دقیق آنها یافت شود.

مثال:

clFoo.h

#ifndef FOO_H

#define FOO_H

typedef struct foo foo;

foo* foo_create (void);

#endif

clFoo.c

#include "clString.h"

struct foo {int x;}

foo* foo_create (void)

{ struct foo foof= {2};

return &foof;}

در مثال زده شده فایل سرایند با استفاده از پیش پردازنده `ifndef` تعریف شده بودن یک شناسه که تعریف شدنش فقط در داخل شرط `ifndef` انجام میپذیرد را مورد بررسی قرار میدهد تا از وجود آن شناسه به عنوان نشانه ای که این سرایند قبلا در برگرفته شده بوده یا نه استفاده نماید. سپس در قالب شرط برای حالتی که `ifndef` برقرار باشد (نشانه `FOO_H` تعریف نشده باشد یا به عبارت دیگر `clFoo.h` در برگرفته نشده باشد) تعریف این سرایند را انجام میدهد.

از جاییکه تغییر در فایل های سرایند ها به معنای کامپایل مجدد تمامی فایل هایی که فایل های سرایند در آنها دربر گرفته شده اند میباشد، برای انعطاف پذیری حداکثری ساختار ها را در فایل سرایند فقط در قالب اعلان قرار داده ایم و تعریف آنرا در فایل متن مرتبط انجام داده ایم.

به علاوه میتوان توابع کوچک تر ماژول که صرفا در توابع بزرگتر در دسترس آن مورد استفاده قرار میگیرند را با تعریف آنها به صورت `static` در فایل منبع ماژولشان، به نوعی کپسوله سازی کرد، تا فقط توسط توابع درون فایل قابلیت استفاده از آنها وجود داشته باشد.

تکنیک های C++ :

علاوه بر امکان استفاده از تکنیک های قبل در C++ برای توابع و ساختار ها (البته با تغییر پسوند فایل ها برای C++ طبیعتا)، قابلیت استفاده از شی گرایی و کلاس برای تعریف ماژول ها استفاده از حوضه های نامی وجود دارد. در نتیجه، با زاویه دید شی گرا میتوانیم عملا برنامه ها را در C++ به شکل زیر ماژولار سازیم.

clFoo.h

#ifndef FOO_H

#define FOO_H

namespace fooNm{

struct foo;

Class Foo;

}#endif

clFoo.c

#include "clString.h"

namespace fooNm{

Class Foo {

struct foo {int x;};

foo* foo_create (void)

{ struct foo foof= {2};

return &foof;}

}

}

در این بخش نیز مانند بخش قبل برای انعطاف پذیری حداکثری صرفاً اعلان کلاس ها در فایل سرایند و تعریف آنها و توابعشان در فایل c. مربوطه در فضای نامی مربوطه انجام پذیرفته است.

مرحله 3: نکات کلی ای در بهینه سازی

ترجیح استفاده از حافظه به صورت آرایه ای

در نزدیک بودن خانه های یک آرایه در برنامه اجازه دسترسی سریع تر به آنها را میدهد و استفاده از آن گزینه بهتری در مقایسه با پیاده سازی تخصیص حافظه پویا با استفاده از لیست های پیوندی است.

استفاده نکردن بی دلیل از چند ریختی

با توجه به اینکه قالب کلی استفاده از چند ریختی معمولاً با فراخوانی توابع اشیای متفاوت با استفاده از اشاره گر ها سر و کار دارد، برنامه در زمان اجرا باید بگردد تا شکل مناسب برای شی که اشاره گر در لحظه به آن اشاره میکند، از تابع چند شکلی را برای آن اجرا کند. اینکار پروسه را به دلیل نیاز برای گشتن کمی کند تر میکند.

بخش دوم: مباحث C/C++

کامنت ها در C++

کامنت های تک خطی

توسط کامپایلر ملاحظه نمیشوند، با // نقطه شروعشان علامت گذاری شده و از نقطه شروع به سمت راست آن خط کلا همه محتوای قرار گیرنده کامنت تلقی میشوند. معمولاً در بالا یا سمت راست کد برای توضیح قرار میگیرند.

کامنت های چند خطی

نقطه شروع آن ها با /* گذاشته شده و تا رسیدن به */ که نقطه پایان آنهاست همه نوشته های ما بین، کامنت تلقی میشوند.

داده ها در C++

ذات کلی داده ها در C/C++

مقادیر مورد استفاده ما در C++/C برای برنامه نویسی در نهایت ذاتا یا از نوع متغیر هستند و یا مقداری ثابت، که در هر دو صورت دوباره سر انجام تحت یکی از انواع اساسی (به غیر از نوع void)، یا ترکیباتی از انواع اساسی (که به آن انواع تعریف شده توسط کاربر میگوییم) قرار خواهند گرفت.

شی ها

"شی یک بخش از حافظه است که دارای نام و نوع تعیین شده میباشد"

در رشته کامپیوتر در سطوح مختلف و از دیدگاه های مختلف تعاریف زیادی برای شی ارائه شده اند. برای برنامه نویسی C/C++ این تعریف نقل قول شده از کتاب Programming principles and practice using C++ نوشته آقای Bjarne Stroustrup بی شک بهترین، جامع ترین و کوتاه ترین این تعریف ها میباشد. لازم به ذکر است که نام شی ها باید استاندارد باشند، در وسط آنها فاصله نباشد از حروف عجیب و غریب و فارسی و @ و & و * و خط کسری و حرکات این چینی در آن خبری نباشد، با شماره شروع نشود و از قواعد نام های استاندارد پیروی کند (به دلیل زمان کم در این نسخه وقت توضیح کامل نیست قواعد نام های استاندارد برای C/C++ را خودتان چک کنید)

متغیر ها

شی هایی هستند که برای حفظ نوع خاصی از مقادیر قابل تغییر که در حین تعریف آنها قبل از نامشان نوشته میشود بهینه شده اند.

ثابت ها

به هر داده ای که قابلیت تغییر آن وجود نداشته باشد، چه حافظه اشغال کند و چه نکند ثابت گفته میشود. ثابت ها دو نوع کلی دارند:

- ثابت های مقداری
- متغیر های ثابت

ثابت های مقداری

صرفاً مقدارهایی از انواع اساسی داده هستند (مانند یک عدد که بصورت مستقیم نوشته و از آن در تقسیمی استفاده شده است). همه مقادیر خام از هر نوع اساسی ثابت تلقی میشوند. شیوه تعریف آنها به دو نوع میتواند باشد.

- استفاده مستقیم از مقدار: حضور عینی آنها در کد و استفاده مستقیم از مقدار عددی (چه اعشاری چه صحیح چه منفی چه مثبت)، کارکتر، آرایه یا رشته کارکتری (همان آرایه کارکتری دلیل اینکه فقط رشته کارکتری و نه رشته و آرایه های نوع دیگر را در بخش مربوط به رشته های کارکتری میفهمید) مورد نظر. این نوع استفاده اصلاً پیشنهاد نمیشود چرا که باعث پایین آمدن خوانایی کد و سخت تر شدن تغییرات اساسی میشود (چراکه اگر بخواهیم همه نقاط استفاده از آن مقدار را تغییر دهیم باید به تعداد دفعات استفاده از آن مقدار را به مقدار جدید عوض کنیم). به دلایل ذکر شده استفاده از نوع بعدی ثابت ها توصیه میشود.
- ثابت های نمادین: این ثابت ها یک نام نمادین که به صورت قرار دادی با حروف تماماً بزرگ نوشته میشود برای یک مقدار تعریف میکنند که بجای آن مقدار قابل استفاده بوده و در کد معادل خود آن مقدار تلقی میشود. اینکار علاوه بر خوانا شدن کد به ما کمک میکند تا در صورتی که نیاز به جایگزینی همه دفعات استفاده از آن مقدار در کد شدیم، صرفاً با تغییر مقداری که برای نام نمادین تعیین کردیم به مقدار جدید، بتوانیم کد را به راحتی به روز رسانی کنیم. این نوع ثابت ها با استفاده از پیش پردازنده #define یا در غالب ثابت enum ها قابل تعریف میباشند.

شیء های ثابت

به ثابت های نام داری که بخشی از حافظه را در تمام عمرشان برای مقدار غیر قابل تغییر اشغال میکنند، شیء های ثابت میگویند. این ثابت ها معمولاً برای آرایه ها و انواع پیچیده داده ها بکار میروند، چرا که از حیث مقداری خاص تر از مقادیر ساده هستند. شیوه تعریف این نوع ثابت ها با قرار دادن `const` قبل از تعریف آنها به شکل یک متغیر میباشد (البته این قضیه در مورد اشاره گر ها کمی پیچیده تر است که در بخش خود به آن نیز میپردازیم). برای شیء های ثابت نیز از اسامی با حروف تماماً بزرگ استفاده میشود. شیء های ثابت باید در زمان تعریف مقدار دهی اولیه شوند.

تعریف شی ها

هر شی فقط یک بار تعریف میشود که همواره و صرفاً به فرمت زیر انجام میشود:

; نام شی نوع داده

تعریف ها در C/C++ دستور بحساب می آیند و پس از هر دستور گذاشتن ; الزامی است.

میتوان تعداد دلخواهی شی از یک نوع (نوع سمت چپ) را با استفاده از , تا قبل از گذاشتن ; مانند فرمت زیر به صورت زنجیر وار تعریف کرد:

; نام شی , نام شی2 , نام شی_دیگر نوع داده

(البته این نوع تعریف کردن در موقع تعریف اشاره گر ها کمی فرق میکند که در بخش مربوطه به آن میپردازیم)

مقدار دهی

در C/C++ مقدار دهی به معنای جایگزین شدن کامل مقدار داخل یک شی با یک مقدار داده شونده میباشد. که این مقدار داده شونده در این زبان در سمت راست قرار دارد. مقدار دهی بین شی های هم نوع یا غیر هم نوع مجاز (که کم هستند و پرداختن به آن به دلیل کم بودن وقت در این نسخه انجام نمیشود)، با استفاده از فرمت زیر صورت میگیرد.

; نام شی/مقدار هم نوع = نام شی گیرنده مقدار

از آنجا که هر مقداردهی نیز دستور به حساب می آید، باید با ; خاتمه یابند.

از لحاظ زمانی مقدار دهی ها کلاً در یکی از دو گروه زیر قرار دارند:

1. مقدار دهی اولیه (در حین تعریف): در صورتی که مقدار دهی در حین تعریف شی صورت گیرد (فرمت زیر پیش آید) به آن مقدار دهی اولیه گفته میشود:

; نام شی هم نوع = نام شی گیرنده مقدار نوع داده

میتوان مقدار دهی اولیه را نیز برای چندین شی هم نوع به صورت زنجیر وار انجام داد:

; نام شی/مقدار هم نوع = نام شی گیرنده , نام شی/مقدار هم نوع = نام شی گیرنده مقدار نوع داده

2. مقدار دهی پس از تعریف: که در ابتدا آنرا دیدید.

دلیل پیدایش این دو گروه این است که برای شیء های ثابت مقدار دهی اولیه تنها دفعه مقدار دهی و برای برخی از انواع پیچیده ای که از ترکیب انواع اساسی داده ها تولید میشوند (مانند آرایه ها و انواع تعریف شونده توسط کاربر) مقدار دهی اولیه به شیء آنها بی دردسر ترین موقع برای مقدار دهی به کل اجزای آنها میباشد. در نتیجه مقدار دهی اولیه به گونه ای، یک فرصت استثنایی است.

انواع اساسی داده ها

انواع اساسی داده ها عبارتند از

- عدد صحیح
- عدد اعشاری
- کارکتر
- void
- bool (مختص C++)

دو نوع اول وابسته به معماری سیستم تعداد متفاوتی بایت از حافظه اشغال میکنند. با قرار دادن `unsigned` در ابتدای تعریف متغیر های از این نوع میزان حافظه را فقط به اعداد مثبت اختصاص میدهند (و این یعنی قابلیت پوشش اعداد مثبت بیشتر). با استفاده از `long` و یا `long long` بعد از تعیین نوع اختصاص حافظه (و قبل از نام نوعشان) نیز میتوان میزان حافظه تخصیص داده شده برای آنها را زیاد نمود.

اعداد صحیح

هر مقدار عددی خام و مستقیم مثبت، منفی یا صفر بدون اعشار در کد های شما برای C/C++ به شکل یک عدد از این نوع دیده میشود. متغیر هایش وابسته به معماری سیستم (اما معمولاً 4 بایت) فضا اشغال میکنند که برای پوشش 2 به توان 32 حالت/تعداد عدد کافی است. نام تعریفی این اعداد در C/C++ ، `int` مخفف `integer` میباشد. مثالی از مقدار دهی اولیه:

```
unsigned int i = 0;
```

```
const using long long int = 2000;
```

اعداد اعشاری

هر مقدار عددی خام و مستقیم مثبت، منفی یا صفر دارای نقطه اعشار مانند 0.0 در کد های شما برای C/C++ به شکل یک عدد از این نوع دیده میشود. نام تعریفی این اعداد در C/C++، float و double میباشد که float معمولا اما شاید نه همیشه، فضای کمتری نسبت به double اشغال میکند. مثالی از مقدار دهی اولیه:

```
float t = 2.4;
```

```
double tt = 3.5;
```

کارکتر

این نوع از داده فضایی به اندازه یک بایت اشغال و برای دربرگرفتن کد های ASCII ارائه میدهد. مقدار دهی عدد به آن ممکن است اما چون این یک بایت برای بازه پوشش مقادیر ASCII ارائه داده شده این که در عکس العمل به عدد های منفی و ... چه میکنند... در کامپایلر با کامپایلر متفاوت است. مقادیری که دریافت میکنند در ' ' قرار میگیرد مثالی از مقدار دهی اولیه:

```
char c = 'c';
```

void

عجیب ترین گزینه در این لیست، این نوع برای تعیین متغیر نیست و صرفا معنای مفهومی دارد. کاربردش تقریبا فقط در توابع و در ارتباط با اشاره گر هاست که در بخش های مربوطه توضیح داده شده اند. اما به طور کلی در توابع به معنی هیچ و در اشاره گر ها به معنی نوع نامعلوم میباشد.

bool

مقدار آن فقط یا true یا false میباشد که در حالت true مقدار عددی داخل آن برابر با 1 و در صورت false مقدار عددی درون آن برابر با صفر است. در صورت دادن هر مقدار عددی غیر صفر به آن خود به خود به یک و حالت true تبدیل میشود و در صورت دادن صفر به حالت false با مقدار صفر تبدیل میشود.

انواع پیچیده داده ها

- اشاره گر
- آرایه ها
- ارجاع ها (مختص C++)
- انواع تعریف شونده توسط کاربر

اشاره گر

یک شی است که بجای مقدار هایی از یک نوع خاص (مانند متغیر های عادی)، برای قرار گیری آدرس حافظه شئی از یک نوع از آن استفاده میشود . قابلیت رفتن و دسترسی به حافظه ی آدرسی که درونش قرار گرفته را دارد.

تعریف آن از فرمت زیر استفاده میکند:

نام_اشاره گر * نوع_شی_ای_که_به_ان_اشاره میکند

معمولا ابتدا آدرس شی مورد نظر که هم نوع با نوع مورد اشاره است به صورت زیر گرفته میشود:

شی_مورد_نظر &

و پس از داده شدنش به اشاره گر در یک مقدار دهی، طی فرایندی که به آن Dereference گفته میشود و فرمت انجام آن به شکل زیر است:

نام_اشاره گر *

به داخل آن آدرس میرود. مثال:

```
int c = 53;
```

```
int * cPtr = &c; \\cPtr yek eshare gar be int ast ke address c be an dade shode
```

```
*cPtr= 4;\\ ba dereference kardan, cPtr be dakhele address c rafte
```

```
\\ va ba meghdar dehi un meghdar dakhel ro avaz kare
```

البته که دسترسی لزوماً به معنی قابلیت تغییر نیست!!! اشاره گر ها همواره میتوانند به آدرس داخلشان بروند اما اینکه قابلیت تغییر مقدار داخل آن را مثل مثال بالا داشته باشند یا نه بسته به درجه دسترسی آنها که در حین تعریفشان مشخص میشود دارد. چرا که دسترسی اشاره گر ها در دوفاز (چهار حالت) قابل تنظیم میباشد:

اشاره گر به آدرس های قابل تغییر اما بدون توانایی تغییر مقدار درون آدرس \\\code{const char * cPtr}; با رفتن به داخل آن\\

اشاره گر به آدرس ثابت اما با قابلیت تغییر مقدار درون آن از طریق رفتن \\\code{char *const cPtr}; به داخل آدرس\\

اشاره گر به آدرس ثابت و بدون قابلیت تغییر مقدار درون آن از طریق \\\code{const char *const cPtr}; رفتن به آدرس\\

اشاره گر به آدرس قابل تغییر و قابلیت تغییر مقدار درون آدرس با رفتن به داخل آن \\\code{char *cPtr};

در صورت اضافه و کسر عدد صحیح به آنها، گام هایی به آن تعداد که هریک به اندازه حافظه ی اشغال شده توسط نوع تعریف شده اش هستند، به آدرس های بالاتر یا کوچکتر بر میدارد.

اشاره گر به void

این نوع اشاره گر صرفاً برای ذخیره آدرس هایی از شئی هایی از انواع نامعلوم استفاده میشود، و قابلیت رفتن به آدرس درونش را ندارد چرا که به دلیل معلوم نبودن نوعش مجاز نیست به انواع متفاوت کاری

داشته باشد، معمولاً با فرایندی که به آن pointer casting یا تغییر نوع اشاره گر میگویند نوع شیء که به آن اشاره میکند متناسب با نوع حافظه ای که در خود جای داده میشود و سپس مورد dereference قرار میگیرد.

شیوه انجام pointer casting:

نام_اشاره_گر (*نوعی که قرار است به آن اشاره کند)

```
void ptr;
```

```
(int*)ptr;
```

اشاره گر به تابع

در ظاهر وحشتناک ترین نوع اشاره گر میباشد و معمولاً برای استفاده غیر مستقیم و شرطی از یکی از چندین تابع مشابه که تعداد، نوع، ترتیب، و نوع بازگشتی شان برابر است استفاده میشود. فرمت آن به شکل زیر است:

(ورودی اول، ورودی دوم)(نام تابع *) نوعی_که_تابع_برمیگرداند

آرایه ها

تعدادی دلخواه و ثابت از شیء های هم نوع بهم چسبیده هستند که با نام آرایه میتوان به هر یک از آنها دسترسی یافت. به تعداد شیء های آرایه سائز آن نیز میگویند. تعریف آرایه ها به صورت زیر میباشد:

[سائز آرایه] نام آرایه نوع داده/نوع آرایه

```
int arr [4];
```

مقدار دهی اولیه آنها میتواند به دو حالت انجام شود:

1- حالتی که در آن سائیز آرایه از پیش تعیین شده باشد (به دلیل سختی نوشتن فعلا به انگلیسی نوشته شد):

```
typename arrayname [size] = { 1st member value, 2nd member value, 3rd member value, ...};
```

```
int arr [4] = {2,3,4,5};
```

2- حالتی که در آن سائیز آرایه بر اساس تعداد مقادیر داده شده به آن توسط کامپایلر شمرده و تعیین میشوند (برنامه نویس وقت برای شمردن ندارد)

```
typename arrayname [] = { 1st member value, 2nd member value, ...};
```

```
int arr [] = {2,5,1};
```

در هر دو صورت در مقدار دهی اولیه هر مقدار به ترتیب از چپ به راست و از متغیر آغازین شروع به مقدار دهی میکند اگر آرایه سائیز از پیش تعیین شده بود تعداد مقدار دهنده ها نباید بیشتر از سائیز آرایه باشند. در صورت کوچک تر بودن تعداد مقدار دهنده ها از سائیز، بقیه شی های آرایه با مقادیر پیش فرضی پر میشوند.

آرایه با استفاده از یک اشاره گر (نام آرایه) که به آغاز اولین شی آنها با کوچکترین آدرس اشاره دارد، قابل پیمایش و اعضایش قابل دسترسی میشوند. چگونگی ممکن شدن این پیمایش به دلیل هم نوع بودن آنها میباشد (چون فاصله نقطه شروع حافظه هر یک از شی ها از دیگری به اندازه میزان حافظه ای است که نوعشان اشغال میکند). از آنجا که اشاره گر به طور عادی در آدرس نقطه ابتدای آرایه است، حالت عادی عملا در "خانه" اول میباشد. بدین گونه ابتدای آدرس شی آخر یا همان "خانه آخر" یکی قبل تر از عدد تعداد کل "خانه" های آرایه است. برای رجوع به هر خانه مجزا از فرمت زیر استفاده میشود:

[شماره خانه] نام آرایه/نام اشاره گر اشاره کننده به ابتدای اولین شی آن

```
arr[3]
```

که عملا معادل فرمت زیر میباشد:

;(شماره خانه + نام اشاره گر) *

مثال:

*(arr + 3)

آرایه های کارکتری

تا اینجا فهمیدیم که آرایه ها از حیث داده ای صرفا اشاره گر هایی به ابتدای عضو آغازین مجموعه هایی با سائز های متفاوت از شی های ممنوع هستند. چند ویژگی و قابلیت مهم و منحصر به آرایه های کارکتری وجود دارد که باعث تمایز آنها از بقیه ی آرایه ها میشود. همه ی آنها نتیجه راهکار هابیست که از زبان اسمبلی در نتیجه استفاده زیاد از کارکتر ها و آرایه های آنها بوجود آمدند.

1- قاعده ی اتمام با null : به دلیل درخواست مکرر برای نمایش نوشته ها و وابستگی به این نوع آرایه برای انجام اینکار، برای مشخص کردن ته متن و نوشته قرارداد بر این شد که حرف پس از حرف آخر آن '0\' باشد (منظور از حرف آخر در اینجا شی آخر نیست بلکه این است که کدام شی حاوی آخرین کارکتر متن ما میباشد). یعنی همواره سائز آرایه باید یکی بیش از تعداد حرف های متن باشد تا برای کارکتر مشخص کننده انتها جا داشته باشد. این حرف '0\' یا حرف null به معنی "حرفی که در جدول ASCII مقدارش صفر است میباشد. (در داخل مقدار کارکتر با نوشتن \ و بلافاصله تایپ چندین مقدار از مبنای عددی 8 تایی کارکتر معادل ASCII آنها به عنوان مقدار تلقی میشود).

2- یک نوع منحصر به فرد مقدار دهی اولیه برای راحتی : برای آرایه های کارکتری نیازی نیست دانه به دانه کارکتر ها به شکل مقابل تعریف شوند:

```
char s[3] = {'s','f','0'};
```

و میتوانند به صورت زیر تعریف شوند:

```
char i[5] = "hi!";
```

که در اینصورت برنامه آنها را معادل حالت قبل و با در نظر گرفتن '0\' پس از حرف آخر مقدار رو به رو بصورت {'0','!', 'I','h'} در نظر گرفته و به i میدهند.

3- قابلیت استقرار اختصاصی بدون نیاز به تعریف متغیر: مقادیری که حالت بین “ ” نوشته میشوند در صورت نداشتن یک آرایه نگه دارنده، با حفظ چینش آرایه ای و بهم چسبیده به صورت خودکار به تعداد شی هایشان برایشان حافظه کنار گذاشته شده و در بخش READONLY حافظه قرار میگیرند. این یعنی برای آرایه های کارکتری حتی نیازی نیست مانند آرایه های بالا تر از فرمتی مثل :

```
char s[] = "Hello";
```

استفاده کنیم و درمورد این نوع آرایه حتی فرمت زیر:

```
char * s = "hello";
```

نیز جوابگو است چرا که “hello” در اینجا خودکار تعداد کارکتر هایش محاسبه و در بخش READONLY مقادیر آرایه ایش ثبت شده اند و s فقط به نقطه اول آن اشاره میکند.

ارجاع ها (مختص C++)

استفاده آنها و مفهوم آنها حقیقتاً تقریباً فقط در حلقه های بازه دار و توابع C++ میباشد در نتیجه در این دو بخش مربوطه به توضیح آنها پرداخته میشود.

انواع تعریف شونده توسط کاربر

این انواع با ترکیب و تلفیق نوع های قبل، تولید شده و به عنوان نوعی برای خود تعریف میشوند. سپس میتوان از آنها درست مانند همه داده های نوع اساسی شیء ساخت.

ساختار ها (نسخه C)

فرمت تعریف آنها:

```
struct blockname {
```

```
int i;
```

```
char s[30];
```

```
{objectName,objarray[30],objectname2;
```

میباشد. فقط تعریف شی در آنها مجاز میباشد. در داخل {} که بدنه ساختار نام دارد، نوع و نام شی های دلخواهی که میخواهیم در نوع باشند تعریف میشوند (در اینجا نمیتوانند مقدار دهی اولیه شوند چون در حال حاضر وجود خارجی ندارند و این فقط تعریف ساختار است). نمونه ها/شی های آن ساختار میتوانند در همان تعریف به تعداد دلخواه از نوع آرایه و اشاره گر به آن ساختار در بعد از بدنه آن با یک , جدا و قبل از ; ساخته شوند. اما اگر بخواهیم بعد از تعریف از آن نمونه/شی بگیریم باید حتما از فرمت زیر استفاده شود:

```
struct blockname object_name;
```

که به دلیل طولانی شدن، معمولا در C بجای نمونه گیری در تعریف (مثل کمی قبل تر) با حذف کلی blockname از یک typedef برای نام دهی به کل تعریف ساختار و در نظر گرفتن کل آن به عنوان یک نوع استفاده میشود:

```
typedef struct{
```

```
int i;
```

```
char s[30];
```

```
}typedef_name;
```

```
typedef_name objectname;
```

اینگونه typedef_name به عنوان نام برای نوع این ساختار قابل استفاده و شی گیری/نمونه گیری صرفا با آن ممکن میشود.

مقدار دهی اولیه به struct ها مانند مقدار دهی اولیه به آرایه ها در {} و به صورت ترتیبی برای اعضایشان میباشد.

```
typedef_name objectname = {مقدار اولین عضو، مقدار دومین عضو};
```

همچنین برای دسترسی به یک عضو از شی ساختار از قاعده:

Objname.membername

استفاده میشود.

اگر دسترسی به عضو یک شی از یک ساختار بخواهد با استفاده از اشاره گری به آن ساختار انجام گیرد، میتوان از فرمت زیر استفاده کرد:

ptrToStructObj ->memberOfStructObjectBeingPointedToInAddress

که معادل فرمت زیر میباشد:

(*ptrToStructObj). memberOfStructObjectBeingPointedToInAddress

مثال:

```
struct foo fl;
```

```
struct foo * fooPtr;
```

```
fooPtr=&fl;
```

در چنین صورتی

```
fooPtr->
```

و

```
(*fooPtr).
```

هر دو معادل "داخل حافظه شی ساختار fl" میباشد.

ساختار ها (نسخه C++)

همه کار های بخش قبل در نسخه C++ ممکن است و فقط یک سری قابلیت های جدید در C++ اضافه و برخی محدودیت ها در آن حذف شده اند که عبارتند:

- ضروری نبودن استفاده از struct در هر نمونه گیری: در C++ دیگر ضرورتی در استفاده از struct برای هر بار نمونه گیری/شی سازی از ساختار نمیباشد (نتیجتاً کل قضیه typedef دیگر لازم نیست). چرا که به صورت خودکار C++ بخش blockname آنرا به عنوان نوع آن در نظر خواهد گرفت.

قابلیت محدود سازی دسترسی با شاخص: با استفاده از public: و private: میتوان تعریف کرد کدام اشیاء و توابع عضو قابل دسترسی مستقیم از طریق شی با همان قاعده objectname.member هستند و کدام یک فقط با توابع درون ساختار قابل دسترسی غیر مستقیم میباشند.

- قابلیت تعریف توابع: همانطور که شنیدید در C++ قابلیت توابع در struct ها وجود دارد که چگونگی انجامشان در بخش توابع توضیح داده میشود.

- وجود توابع سازنده و مخرب: هر بار که یک شی از ساختاری ساخته میشود قبل از هر چیز تابع سازنده آن ساختار اجرا میشود این تابع تابعی همنام با نام نوع ساختار میباشد که در صورت تعریف توسط کاربر میتواند پارامتر بگیرد و بدنه اش دسترسی مستقیم به همه اعضای ساختار دارد، اما حق بازگردانی مقداری را ندارد. در صورت تعریف نشدن توسط کاربر یک سازنده به نام سازنده پیش فرض در پشت صحنه تولید میشود که در بدنه خود، سازنده یا سازنده پیش فرض شی های بدون مقدار پیش فرض عضو خود را صدا میکند. در حین خروج از بازه وجود شی تابع مخرب صدا زده میشود که تابعی بدون مقدار ورودی یا خروجی و وابسته به نام آن نوع میباشد. این تابع نیز مانند سازنده میتواند توسط کاربر تعریف شود که در اینصورت باید از فرمت زیر استفاده کند:

{بدنه} (نام_نوع_ساختار)~

و در صورت تعریف نشدن به مخرب پیش فرض تولید میشود که رویه ای مانند سازنده را اما برای مخرب ها طی میکند.

- امکان ارث بری: در C++ امکان ارث بری ساختارها از هم و کلاس ها و بلعکس وجود دارد. ارث بری به معنی حضور تمام بخش های کلاس یا ساختار ارث بری شونده در ساختار یا کلاس ارث بری کننده میباشد. فرمت کلی در حین تعریف نوع تعریف شونده توسط کاربر ارث بری کننده انجام میشود:

```
struct Derived : access_modifier Base{};
```

در اینجا access-modifier تعیین کننده نوع دسترسی و ارث بری است (بله ارث بری انواع زیادی دارد که فعلا به دلیل کمبود وقت به آن پرداخت نمیشود)، Base ساختاری است که از آن ارث بری میشود و Derived کلاس در حال تعریف است که از Base ارث بری میکند.

یونین ها (نسخه C)

به غیر از دو تفاوت که گفته میشوند دسترسی و استفاده آنها دقیقا عین ساختار های C است.

تفاوت اول: بجای struct از union حین تعریف و نمونه گیری آنها استفاده میشود.

تفاوت دوم: میزان حافظه اعضایشان هم پوشانی دارد و حافظه ای به اندازه بزرگترین عضویشان را با هم شریکند، تغییر در مقدار بزرگترین عضو کل آن حافظه را عوض و تغییر در میزان اعضای کم حافظه تر فقط آن بخش از حافظه کلایشان را عوض میکند. کاربرد آنها بیشتر در کنار بیت فیلد ها معنا میابد.

یونین ها (نسخه C++)

همه قابلیت های ساختار های C++ را دارند به غیر از قابلیت ارث بری و داشتن توابع virtual و چند ریختی.

بیت فیلد ها

داخل ساختار ها و یونین های C/C++ امکان تعریف نوع ای خاص به نام بیت فیلد وجود دارد که فقط در اینجا میتواند تعریف شود و حقیقتا استفاده از آن بیشتر در یونین ها و به منظور استفاده و تحت نظر قرار دادن حافظه اشتراکی توسط بازه هایی از بیت ها میباشد. این نوع داده برای تعیین تعدادی بیت بهم چسبیده استفاده میشود که سپس با توجه به گونه خوانش تعیین شده (علامت دار یا بدون علامت) به عنوان مقادیر عددی به ما خروجی داده میشود (مسئله نوع خوانش بیشتر مربوط به حوضه و بخش مدار منطقی و اسمبلی است که قرار بود در این نسخه پایان نامه در کنار اسمبلی x86 توضیح داده شوند اما بدلیل زیغ وقت در این نسخه نیستند).

اما گونه تعریف آن درون بدنه، ساختار یا یونین و در تعریف آن انجام شده و به شکل صفحه بعد است:

بدنه ساختار یا یونین //}

; تعداد بیت : نام_متغیر int signstatus

}

که در اینجا بجای signstatus باید unsigned برای حالت ترجمه به عنوان عدد بی علامت بیت ها، فضای خالی برای حالت ترجمه به عنوان عدد علامتدار بیت ها قرار بگیرد. int نیز همواره در تعریف بیت فیلد ها ثابت و باید حضور داشته باشد.

کلاس ها (مختص C++)

تقریباً از نوع کاربردی کاملاً عین ساختار های C++ هستند با این تفاوت جزئی که ساختار های C++ به طور پیش فرض مقادیرشان به طور مستقیم قابل دسترس بوده در حالیکه کلاس ها بطور پیش فرض مقادیرشان غیر قابل دسترسی مستقیم است. فرمت تعریف آنها به شکل زیر است:

; {توابع و سازنده و مخرب و داده ها} نام کلاس class

تابع ها

تابع ها مجموعه هایی از دستورات شرطی ، غیر شرطی، مقداردهی ها و روال ها هستند که میتوانند وابسته به ورودی هایی از بیرون باشند، و مقادیری به بیرون بازگردانند. به ورودی های توابع در صورت وجود پارامتر میگویند. در صورت نبود هیچ ورودی یا خروجی به جای آن void میگذاریم. تعریف توابع به طور کلی به فرمت صفحه بعد است:

(نام_پارامتر نوع، نام_پارامتر نوع) نام_تابع نوع_مقداری_که_بازمیگرداند

```
{
```

دستورات بدنه

نام_شیء_یا_ثابت_همنوع_مقدار برگشت return

```
}
```

درمورد فرمت توابع و فراخوانی آنها منابع زیادی به اندازه کافی توضیح داده اند به دلیل کمبود وقت به سراغ نکات مهمتر میرویم. مشخصات اساسی هر تابع از قبیل نوع مقدار بازگردانی کننده، نام و تعداد و ترتیب و نوع پارامتر معمولاً در همان خط اول تعریف آن میباشد. این مشخصات باید قبل از استفاده فراخوانی های تابع در حوضه مورد استفاده یا حوضه والد آن شناخته شده باشند.

حوضه ها

حوضه عمومی

والد همه حوضه ها میباشد. زمانی که داده بدون قرار گرفتن در تابع دیگری و کلمه کلیدی static در حین تعریف آن در داخل یک فایل کد منبع تعریف شود در حوضه عمومی میباشد. تابع های C اکثراً در حوضه عمومی میباشد. دسترسی به توابع و اعلان های حوضه عمومی از حوضه های زیر دست امکان پذیر میباشد و دسترسی به داده های این حوضه در صورت نبود شی های همنام در حوضه زیر دست، به طور مستقیم در C ممکن است.

نکته C++: در این زبان برای دسترسی به شی عمومی از حوضه زیر دست میتوانید از فرمت نام_شی :: استفاده نمایید.

حوضه namespace (مختص C++)

با فرمت صفحه بعد تعریف میشود:

```
namespace نام{
```

```
    کلاس ها و توابع
```

```
}
```

برای دسترسی به کلاس ها و توابع داخل باید از `using namespace` یا `namespace::` استفاده کرد. میتوان تفاوت های آنها را جستجو کنید به دلیل کمبود زمان به آن پرداخت نمیشود.

حوزه کلاس (مختص C++)

داده ها و توابع تعریف شده در آن فقط از داخل شیء یا به طور مستقیم یا به طور غیر مستقیم قابل دسترسی اند.

در صورت تعریف شدن از نوع `static` یکبار به طور کلی تعریف شده و بدون داشتن شیء از آن کلاس قابل دسترس خواهند بود اما به همین دلیل نمیتوانید در آنها به مقدار دهی داده ها های وابسته به وجود شیء بپردازید. چون برخلاف این نوع `static` هنوز وجود ندارند، مگر اینکه مقدار دهی بعد از ساختن شیء انجام شده باشد. علاوه بر این داخل توابع عضو کلاس ها میتوان از یک اشاره گر `this` استفاده کرد که آدرس خود شیء را برمیگرداند و طبیعتاً این اشاره گر نیز در توابع `static` قابل استفاده نیست چون فارغ از شیء و بدون آن وجود و فعالیت میکنند.

برای ورود به حوزه کلاس باید از فرمت `:: نام_کلاس` استفاده میشود. این قضیه برای ساختاری `C++` نیز کاملاً برقرار میباشد.

حوزه فایل

داده ها و توابع درون یک فایل با استفاده از `static` در قبل از تعریفشان به این حوزه محدود میشوند یعنی فقط توسط سایر توابع و ... های آن فایل قابل دسترسی میشوند.

حوزه تابع

داده های این حوزه فقط در درون تابع قابل دسترسی اند و به آنها داده های محلی میگویند. اگر به صورت `static` تعریف شوند مقادیر خود از دفعه آخر فراخوانی حفظ میکنند.

در نهایت دوست داشتم بیشتر به این بخش و storage class ها بپردازم ولی واقعا باید تا 7 ساعت دیگر این نسخه فعلی را ارائه دهم. در نتیجه به توضیح اعلان ها میپردازیم.

اعلان ها

گاهی اوقات به دلایل گوناگون میخواهیم از یک تابع یا شیء که تعریف آن در جای دیگه انجام داده شده استفاده کنیم. به این منظور باید اعلان مربوط به آن قبل استفاده از آن داده شود.

اعلان ها با توجه به پیچیدگی داده دو دسته اند:

1- اعلان داده های غیر پیچیده

2- اعلان های داده های نوع پیچیده (همان نوع تعریف شده توسط کاربر)

اعلان های غیر پیچیده

اعلان های غیر پیچیده

- اگر تابع باشد در اینصورت تا پایان بخش تعریف پارامتر های تابعش که در فایل کد منبع دیگر است را کپی میکنیم و پس از آن یک ; میگذاریم. اینکار باید قبل از فراخوانی از آن تابع در فایل فعلی مان و در حوضه عمومی انجام شود.

- اگرشی باشد از کلمه کلیدی extern استفاده میکنیم و کامپایلر مسئولیت یافتن آنرا به لینکر میسپارد و لینکر آنرا حل میکند.

اعلان های داده های نوع پیچیده (همان نوع تعریف شده توسط کاربر)

از بخش تعریفشان فقط تا بخش نام آنها را جدا کرده و در حوضه عمومی فایل مورد استفاده آن قرار میدهم و با گذاشتن ; پس از آن یک اعلان میشود که تعریف آن در فایل مربوطه قرار دارد.

پیش پردازنده ها

این دستورات همانطور که از اسمشان مشخص است قبل از رخداد فرایند کامپایل انجام شده و برای رسیدگی اولیه به اینکه مقادیر کلی چه باشند هستند. از پر کاربرد ترین آنها عبارتند از:

`#include`: برای دربر گرفتن فایل های سرایند در آدرس نسبی از فایل کد منبعی که در آن قرار نوشته میشود.

`#define NAME replacementcode`: پس از دیدن چنین کدی با هربار دیدن `NAME` کل `replacementcode` را به جای آن قرار میدهد.

این پیش پردازنده قابلیت تابعی نوشته شدن را دارد که به آن ماکرو نیز میگویند و حقیقتاً توضیح آن واقعا وقت گیر است اما شکلی بسیار پر کاربرد در کنار پیش پردازنده های `#if`، `#elif`، `#ifdef`، `#ifndef` میباشد. متأسفانه به دلیل وقت فرصت توضیح آن در این نسخه فراهم نشد.

نتیجه گیری کلی

در این نسخه از پایان نامه تمام تلاش خود را کردم تا با ارائه دادن پروژه هایی جالب اما قطعا بسیار ساده از زاویه برنامه نویسی، شاید علاقه مندان به زیر شاخه هایی از برنامه نویسی را جذب و امیدوارانه مطالبی به آنها آموخته باشم.

نکاتی که وقت برای پرداختن به آنها نرسید

با توجه به زمان و شرایط در زمان نوشتن این نسخه از پروژه موفق به پرداختن و پوشش نهایتا یک دهم آنچه در نظر داشتم، شدم. فهرستی از گزینه هایی که قرار بود به آنها پرداخت شود را در بخش زیر میبینید.

در پیوست 1:

آموزش مباحث مدار های منطقی

قصد داشتم موضوعاتی نظیر مبنا های عددی را به طور کامل توضیح دهم که وقت اجازه اینکار را نداد.

پرداختن به شبکه های کامپیوتری

توضیح شیوه کار کلی و دغدغه هایی کلی در حین پیاده سازی شبکه های کامپیوتری.

در پیوست 2:

توضیح فوق عمقی کلاس ها

پلیمورفرسیم یا چند ریختی

به استفاده از خصوصیات ارث بری برای پیاده سازی متفاوت یک تابع در کلاس های ارث پری کننده متفاوت میپردازت.

سربارگزاری اپراتورها

به تغییر معنی اپراتور ها در مواجه با شی هایی از یک کلاس میپردازت. چگونگی کار `std::cout` به دقت قابل درک میشد.

تجمیع در کلاس های C++

استفاده از یک شی از یک کلاس در داخل کلاسی دیگر به عنوان یک عضو و دردرس ها و دغدغه هایی که میتواند به همراه بیاورد.

پرداختن به حلقه ها و دستورات کنترلی

آموزش اصول اساسی تر و پر استفاده و برخی ارور های مرتبط و رایج و راه های مقابله با آنها و فعالیتشان.

پیوست 3 برای آموزش Assembly x86

قرار بود پیوست سومی هم برای آموزش کامل x86 برای معماری های 64 و 32 بیت با اسمبلر MASM پرداخته شود اما دغدغه زمان آنرا ناممکن ساخت.

پیوست 4 آموزش WinAPI

قرار بود که پیوست چهارمی نیز برای توضیح کار با API ویندوز و ساخت پنجره با آن و دادن آیکون به پنجره و آموزش زیر ساخت های آن باشد که باز بخاطر زمان و شرایط ممکن نشد.

پیشنهادهات

پیشنهاد میشود اگر به دنبال یادگیری عمقی مطالب گفته شده هستید نگاهی به کتاب های منبع و فهرست منابع معرفی شده ام بیاندازید. همچنین در صورت پرسش سوال مرتبط میتوانید آنرا به آیدی تلگرام : t.me/Anti0bcel بفرستید.

فهرست منابع

- Andriesse, D. (n.d.). *Practical Binary Analysis Build Your Own Linux Tools for Binary Instrumentation, Analysis, and Disassembly*.
- Dietel, P., & Dietel, H. (n.d.). *C How to Program (9th Edition)*.
- Dietel, P., & Dietel, H. (n.d.). *C++ How to Program (Tenth Edition) (Global Edition)*.
- Irvine, K. (n.d.).
- Kernighan, B. W., & Ritchie, D. M. (n.d.). *The C Programming Language (ANSI C)*.
- Petzold, C. (n.d.). *Programming Windows The definitive guide to Win32 API (5th Edition)*.
- Sellers, G., Wright, R. S., & Haemel, N. (n.d.). *OpenGL Superbible (Seventh Edition)*.
- Shreiner, D., Sellers, G., Kessenich, J., & Licea-Kane, B. (n.d.). *OpenGL Programming Guide (Eight Edition)*.
- Sikorski, M., & Honig, A. (n.d.). *Practical Malware Analysis*.
- Stroustrup, B. (n.d.). *Programming Principles and Practice Using C++ (Second Edition)*.



Final Project

Bachelor of Science in Software Engineering

Technical & Engineering Faculty

(Department of Computer Engineering)

Graphics Programming Concepts and Implementation

Submitted by: Amirmohammad Khaleghi Farid

Supervisor:

Omid Aqa Latifi

[Dec 2025]